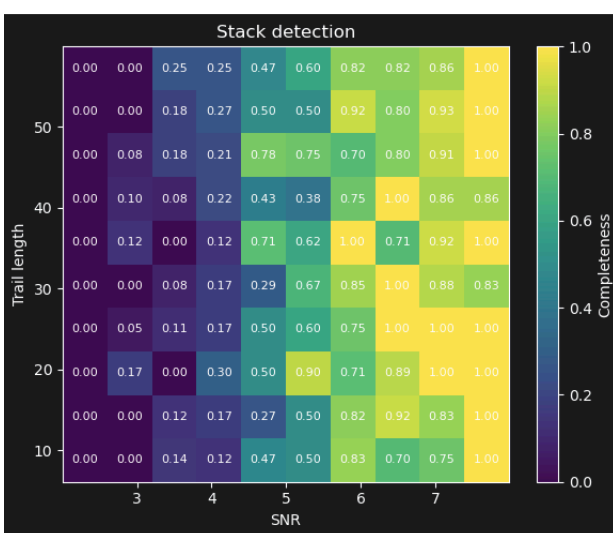
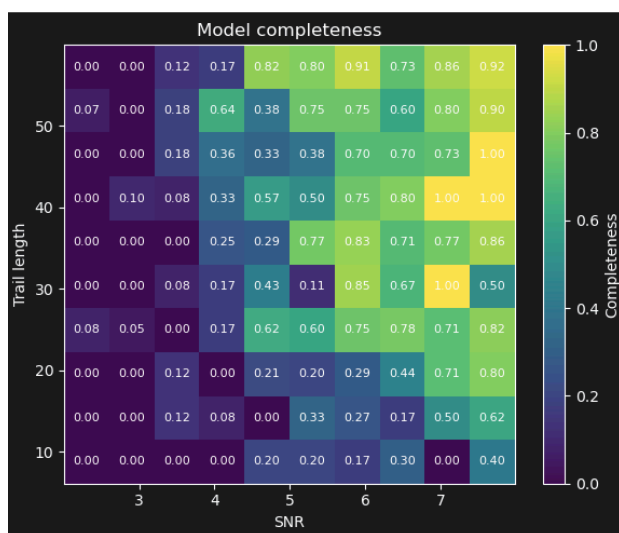
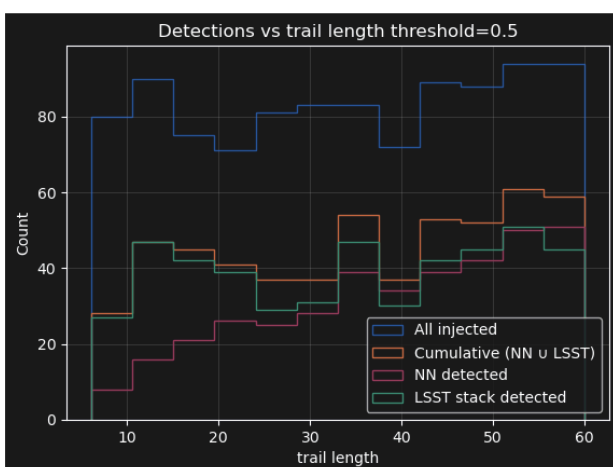
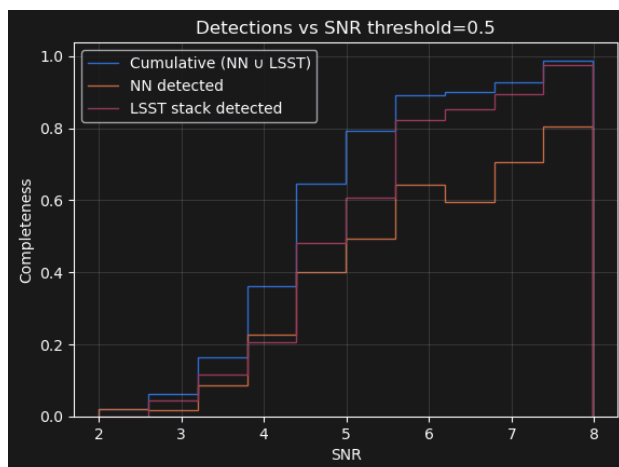


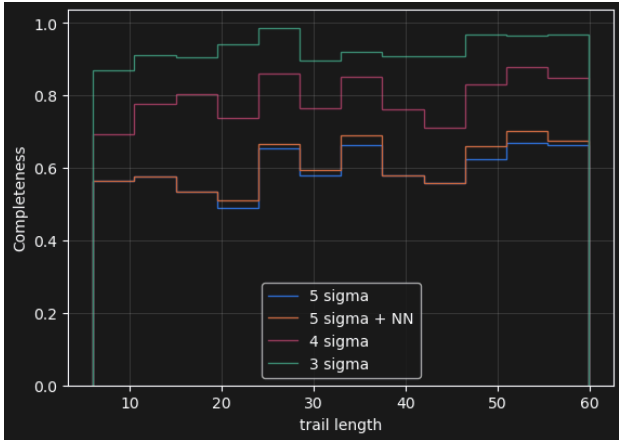
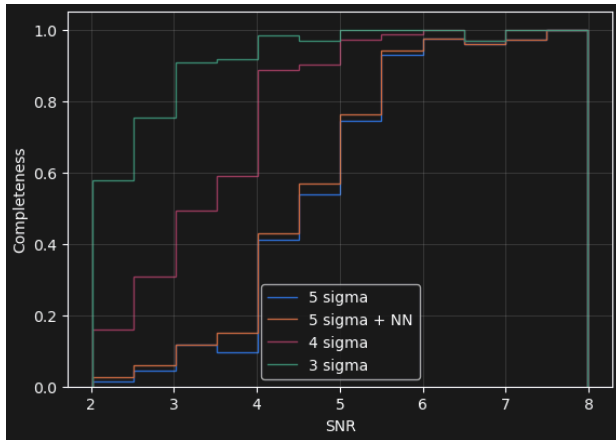
May 11, 2026 | 📅 LSST Streak Detection Weekly

I pivoted on training and testing on the difference images, these are the results.

Sigma		Only NN			NN with Stack		
	Method	Missed	Recovered	Extra detections	Missed	Recovered	Detections of not asteroids
5	NN (t=0.97)	675	325	337	464	536	6742
	NN (t=0.5)	621	379	2097	449	551	8502
	Two-stage NN	608	392	2519	448	552	8924
4	NN (t=0.97)	688	312	259	301	699	15288
	NN (t=0.5)	619	381	2053	296	704	17082
	Two-stage NN	601	399	2476	293	707	17505
3	NN (t=0.97)	633	367	291	116	884	142363
	NN (t=0.5)	579	421	1834	113	887	143906
	Two-stage NN	559	441	2229	112	888	144301

Stack detections			
Sigma	Missed	Recovered	Detections of not asteroids
5	525	475	6405
4	318	682	15029
3	121	879	142072





Apr 6, 2026 | LSST Streak Detection Weekly

The original intuition behind the threshold scan was that lowering the direct-image detection threshold from 5σ to 3σ should produce a very steep, nearly exponential increase in the number of detected sources, similar to the behavior expected for random-noise fluctuations or for

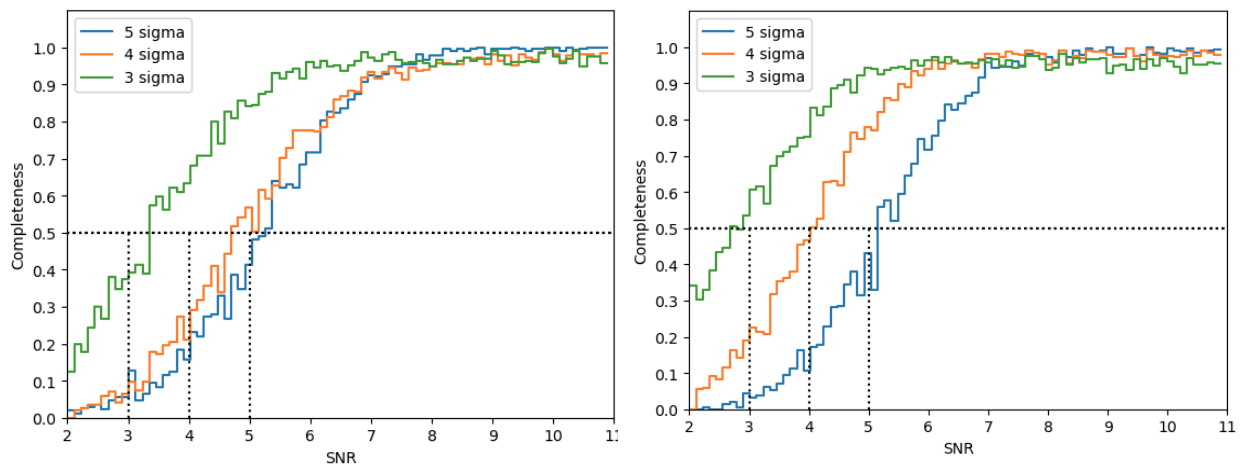
difference-image detections. After the Slack discussion and follow-up experiments, this interpretation no longer appears to be correct for single-visit direct images. The key point is that Eq. 7 from [Jones et al.](#) describes the scaling of random-noise peaks, while the source counts measured on a single-visit direct image are dominated by real astrophysical objects, blends, and structured background, not by pure Gaussian noise.

Lowering the threshold from 5σ to 3σ corresponds to only about 0.55 mag in depth. If astrophysical sources dominate and follow typical number-count slopes ($k \approx 0.5$), one expects only a modest increase in detections, of order a factor of ~ 2 . The measured increase (1261 $\rightarrow$ 3927 detections) is therefore not anomalous and is consistent with physically reasonable behavior of the stack, even if the detailed noise properties are still not fully understood.

This changes the interpretation of the experiment. The earlier discrepancy is no longer evidence of a bug or hidden suppression in SingleFrameDetectAndMeasureTask. Instead, the stack behavior appears broadly consistent with expectations, and the initial intuition was based on an incorrect comparison between direct-image detections and noise-only or difference-image arguments.

This has direct consequences for the neural-network results. One motivation was that the 5σ stack might be underperforming, allowing a neural network to recover “missing” detections. The current evidence does not support that scenario. Lowering the stack threshold itself is already a very strong baseline.

The neural network clearly learns useful signal and can recover objects missed by the 5σ stack, but it does not yet demonstrate superiority over simply lowering the detection threshold. The gain from NN + 5σ stack is real, but becomes less compelling when compared to 4σ or 3σ stack performance. At present, there is no convincing evidence that the NN provides a better completeness vs. candidate-load tradeoff than classical threshold tuning.



Left is “stdev” threshold type and right is “pixel_stdev” on 50 detector point source injections. Below are numbers from 2 detectors from different visits, number of source detections without any injections. Clearly exponential growth is not present.

theory 5 σ noise detections	theory 4 σ noise detections	theory 3 σ noise detections
2.06224	148.509573	3688.471217

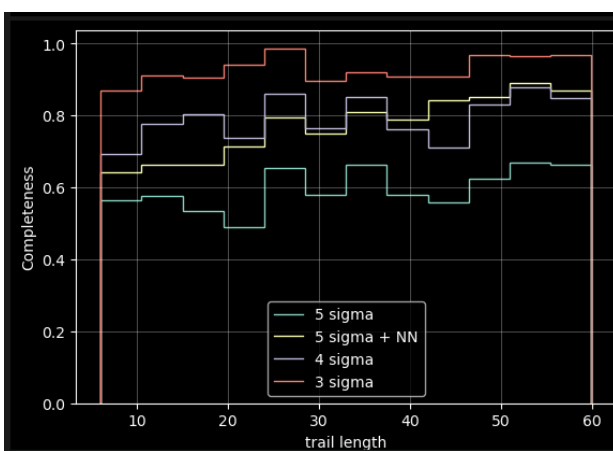
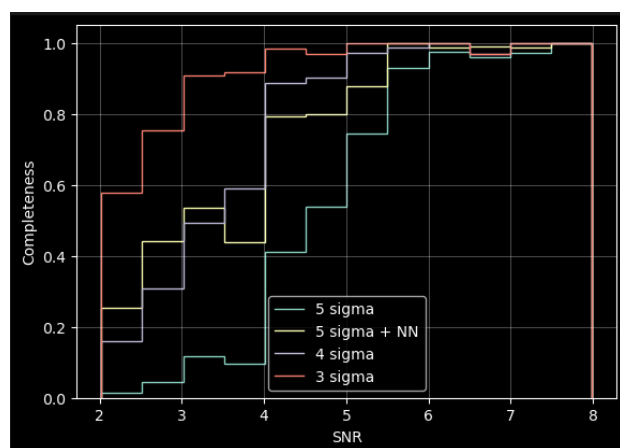
reEstimateBackground	doTempLocalBackground	filter_bad_flags	thresholdType	measured 5 σ	measured 4 σ	measured 3 σ
True	True	True	pixel_stdev	2890	3686	5732
True	True	True	stdev	2401	2845	4166
True	True	False	pixel_stdev	4006	5642	4166
True	True	False	stdev	3249	3931	10053
True	False	True	pixel_stdev	2889	3682	6299
True	False	True	stdev	2401	2844	5731
True	False	False	pixel_stdev	4087	5887	4164
True	False	False	stdev	3273	3978	10644
False	True	True	pixel_stdev	2890	3686	6497
False	True	True	stdev	2401	2845	5732
False	True	False	pixel_stdev	4006	5642	4166
False	True	False	stdev	3249	3931	4166
False	False	True	pixel_stdev	2889	3682	10053
False	False	True	stdev	2401	2844	6299
False	False	False	pixel_stdev	4087	5887	5731
False	False	False	stdev	3273	3978	4164

Mar 30, 2026 | 📅 LSST Streak Detection Weekly

These results are same run from last week only making sure same visits and detectors are used for different sigma thresholds and injections are in the same place.

Sigma		Only NN			NN with Stack		
	Method	Missed	Recovered	Extra detections	Missed	Recovered	Detections of not asteroids
5	NN (t=1.0)	390	610	32618	261	739	149367
	NN (t=0.5)	329	671	44998	222	778	161747
	Two-stage NN	331	669	40553	224	776	157302
4	NN (t=1.0)	390	610	27461	163	837	180395
	NN (t=0.5)	332	668	39861	143	857	192795
	Two-stage NN	332	668	35516	143	857	188450
3	NN (t=1.0)	410	590	20579	61	939	289435
	NN (t=0.5)	353	647	31838	53	947	300694
	Two-stage NN	353	647	27756	53	947	296612

Stack detections			
Sigma	Missed	Recovered	Detections of not asteroids
5	405	595	116749
4	207	793	152934
3	70	930	268856

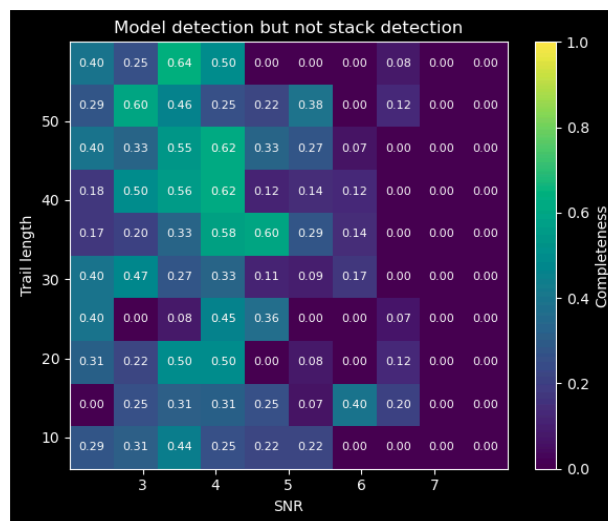
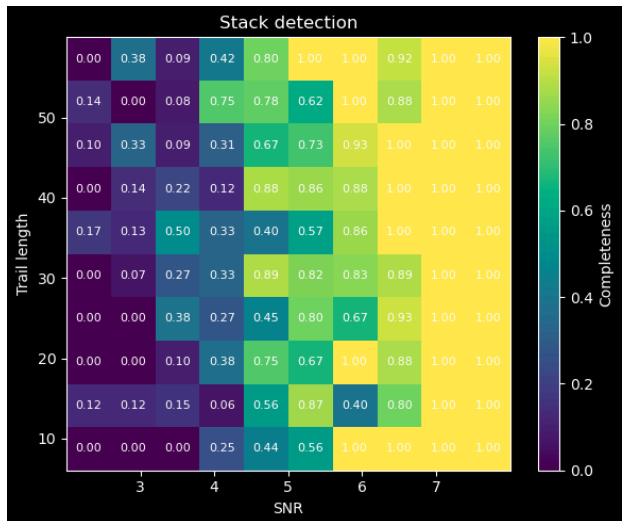
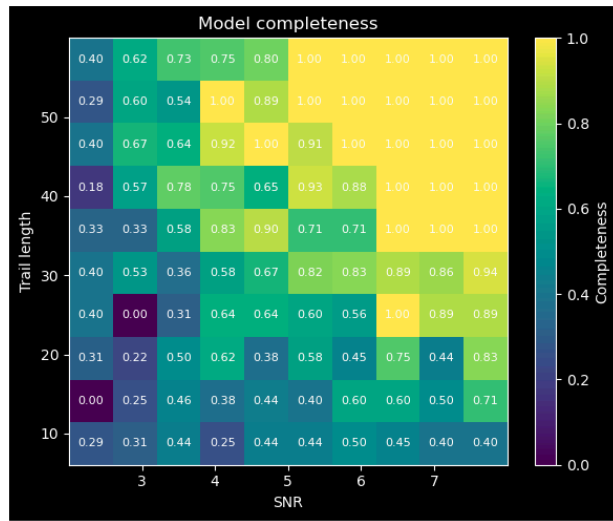
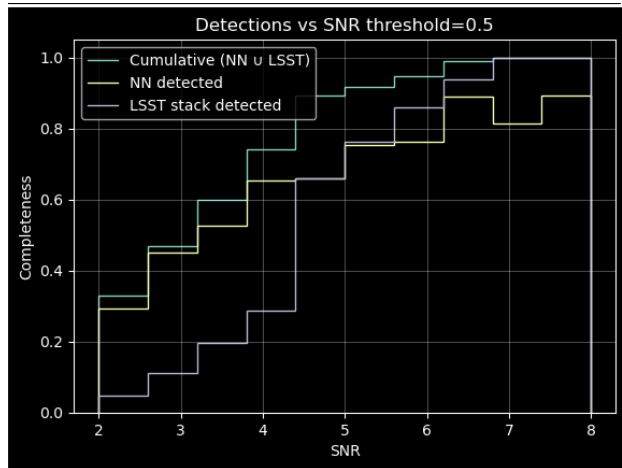
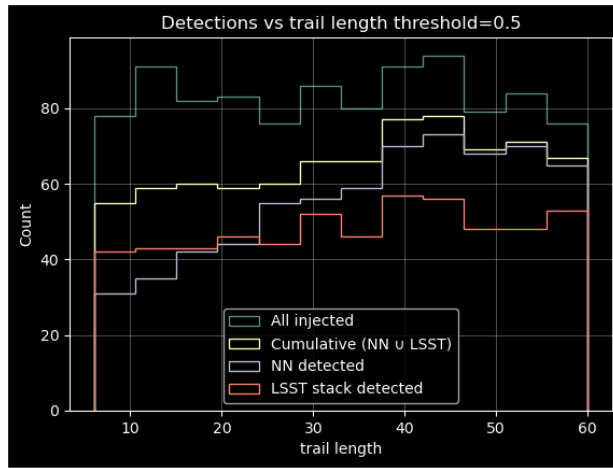


Sigma		Only NN			NN with Stack		
	Method	Missed	Recovered	Extra detections	Missed	Recovered	Detections of not asteroids
5	NN (t=1.0)	396	604	32468	254	746	159678
	NN (t=0.5)	330	670	44215	210	790	171425
	Two-stage NN	333	667	39852	213	787	167062
4	NN (t=1.0)	416	584	27335	185	815	189928
	NN (t=0.5)	347	653	39177	158	842	201770
	Two-stage NN	348	652	31014	158	842	197501
3	NN (t=1.0)	433	567	19534	93	907	312947
	NN (t=0.5)	371	629	30374	85	915	323787
	Two-stage NN	379	621	22864	85	915	319804

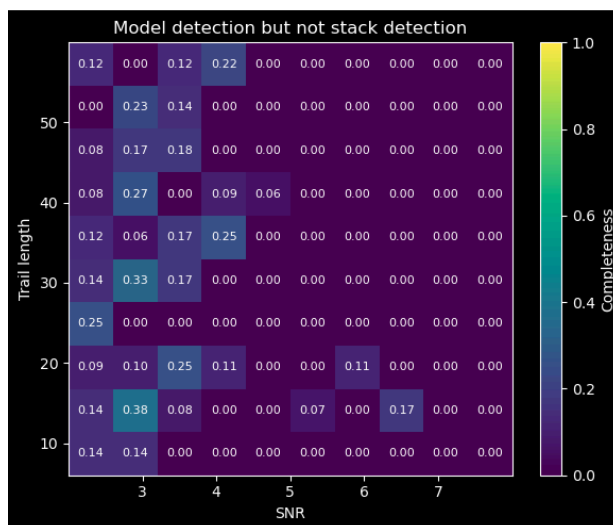
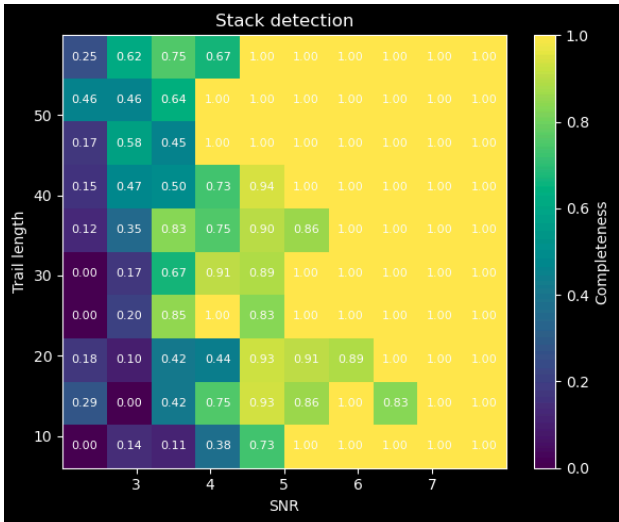
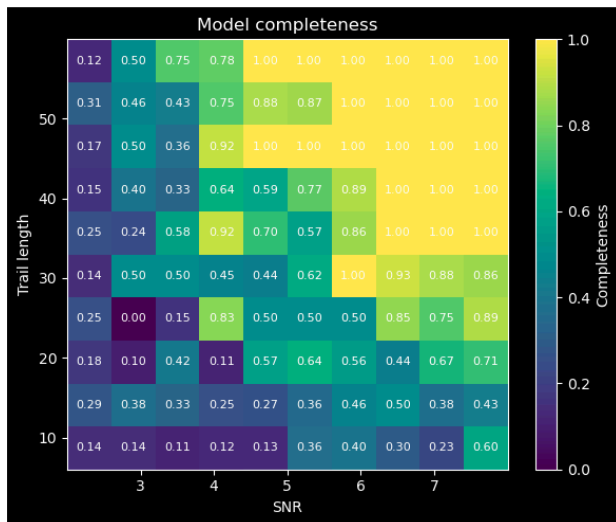
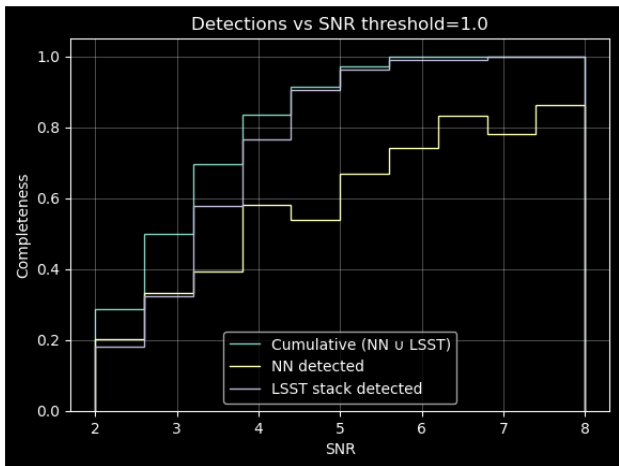
Stack detections			
Sigma	Missed	Recovered	Detections of not asteroids
5	422	578	127210
4	239	761	162593
3	102	898	293413

The neural network does contain useful detection signals and can recover a substantial number of injected objects missed by the standard 5σ LSST stack, but it does not yet clearly outperform the simpler strategy of lowering the classical stack detection threshold itself. The combined 5σ stack + NN improves completeness significantly relative to the standard 5σ stack alone, meaning that the model is learning something real and astrophysically relevant, but similar gains can also be obtained by relaxing the stack threshold to 4σ or 3σ . The results do not demonstrate a decisive advantage in a better completeness-versus-candidate-burden tradeoff than threshold tuning of the stack alone.

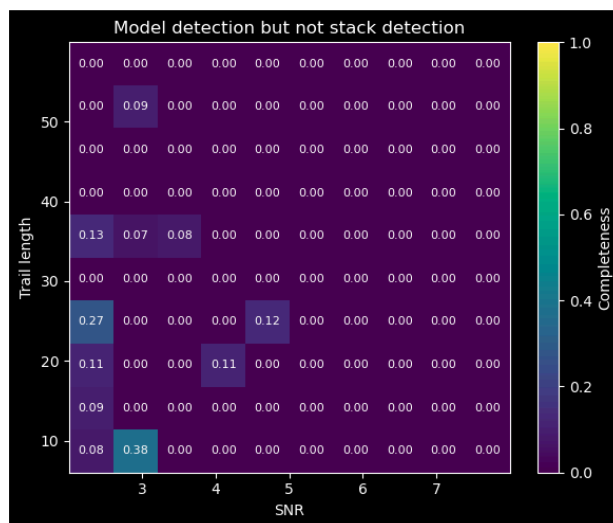
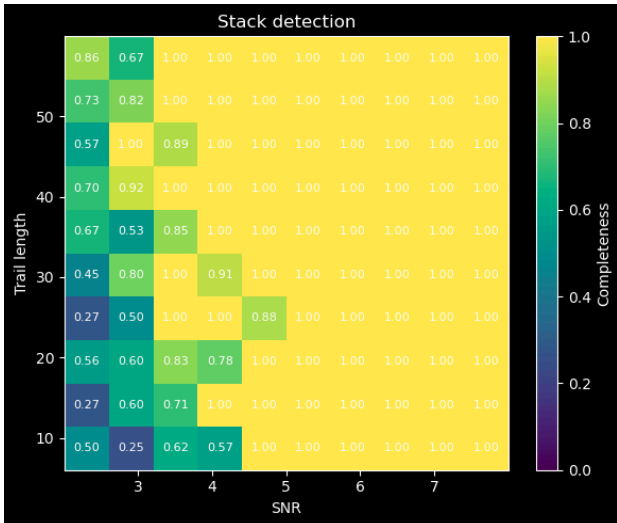
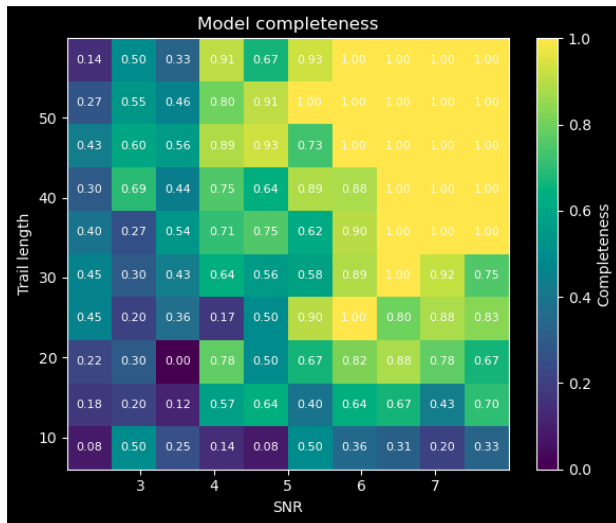
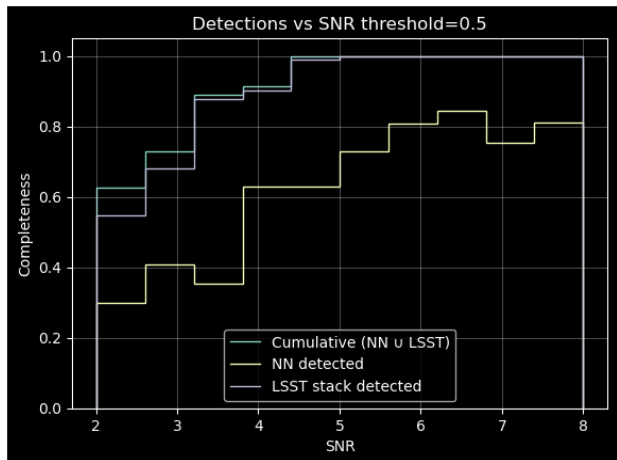
5 sigma



4 sigma



3 sigma



I run the final evaluation of the trained model on the 50-detector test set and compare it directly to the LSST stack at different single-frame source-detection thresholds. The main result is that the neural network is most useful when the LSST stack is kept at its standard 5-sigma operating point. In that case, the stack alone detects 578 of 1000 injected objects, while the neural network alone detects 604 at its best threshold, and the combined LSST + NN detection reaches 746. This shows that the model can recover a substantial number of sources missed by the standard 5-sigma stack detector.

At the same time, the evaluation shows that this gain must be interpreted together with the false-positive burden. The LSST stack itself already produces a very large number of false positives, and the neural network adds its own additional detections depending on threshold choice. This means that the relevant question is not simply whether the NN can detect more objects, but whether it can improve cumulative detection in a useful way without making the candidate set unmanageable. In that sense, the NN is better understood as a completeness-boosting stage on top of the stack, not as a clean replacement for it.

A second important result is that the usefulness of the neural network depends strongly on how aggressive the LSST stack detector already is. When the stack threshold is lowered from 5 sigma to 4 sigma, the stack alone detects 761 of 1000 injected objects and the combined LSST + NN result increases only to 815. When the stack threshold is lowered further to 3 sigma, the stack alone detects 898 of 1000 and the combined result reaches only 907. This means that the model adds the most value when paired with the standard 5-sigma stack, while its incremental benefit becomes much smaller once the classical detector is already run more aggressively.

Mar 18, 2026 |  LSST Streak Detection Weekly

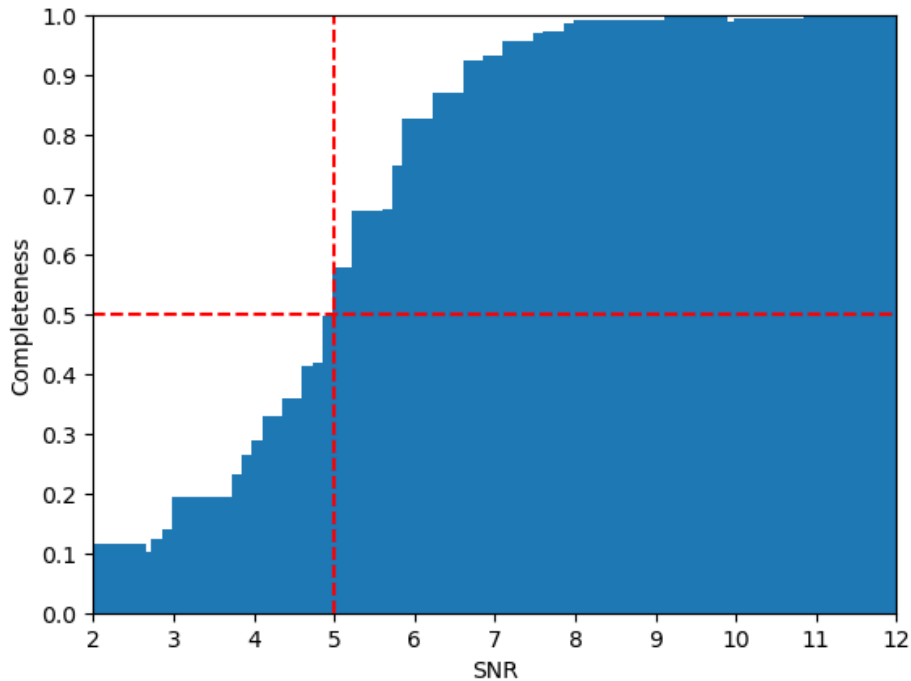
Reproducing the official Butler catalog in Python

The immediate goal is to reproduce, in Python and for a single detector, exactly the same single-frame catalog that is already stored in the Butler collection. With Ian Sullivan's help, I am able to reconstruct the modern LSST pipeline path based on CalibrateImageTask and recover the same source counts as the stored Butler product.

This is an important validation step. It shows that the discrepancy identified earlier is not caused by a hidden Butler-side transformation or a storage issue, but by the fact that I had previously been running the wrong task sequence in the notebook. Once the task graph is reconstructed correctly, the Butler catalog and the Python-produced catalog agree. This confirms that I now understand how to reproduce the official LSST single-frame DRP product from within Python.

Validating the setup with point-source injections

After reproducing the official DRP catalog, I run a direct validation test using injected point sources. The goal is to make sure that the end-to-end setup is internally consistent before moving on to more specialized faint-trail experiments.



In this validation, I inject point sources spanning a range of SNR values and measure completeness as a function of injected SNR. The resulting completeness curve behaves as expected: recovery rises monotonically with increasing SNR, crosses about 50% near $\text{SNR} \approx 5$, and approaches nearly 100% at higher SNR. This is an important sanity check, because it shows that the injection, rerun of the pipeline, and recovery logic are all working in a physically reasonable way for ordinary point sources. In other words, I am now able to reproduce the expected SNR-versus-completeness behavior on point sources, which gives confidence that the basic machinery is correct.

What this does and does not mean

At this stage, however, an important subtlety becomes clear. Even though I can now reproduce the official LSST DRP catalog and obtain a reasonable SNR-versus-completeness curve for injected point sources, that still does not mean that the returned catalog should be interpreted as a generic “5 sigma direct-image detection catalog.”

The reason is that the official DRP single-frame catalog produced by CalibrateImageTask is a **calibration-oriented catalog**. It is primarily designed for PSF modelling, astrometry, and photometric calibration, not to provide the most complete possible list of all astrophysical sources down to the nominal single-visit limiting depth.

Why the DRP catalog is more restrictive than a generic 5σ source catalog

The key result of this stage is that, in the DRP path, the relevant direct-image catalog is intentionally more restrictive. The `single_visit_star_*` / `single_visit_star_unstandardized` family produced by CalibrateImageTask is not meant to be a science-complete direct-image source catalog. Instead, it is designed to provide a clean and reliable set of sources for calibration. The runtime logs showed that, even when the exposed configuration appeared to request a nominal 5σ threshold, the relevant final detection stage inside CalibrateImageTask was effectively using `thresholdValue=5.00`, `multiplier=2.0` corresponding to an effective threshold of approximately 10σ for the returned `single_visit_star_*` catalog.

This explains an earlier ambiguity. Seeing a nominal “5 sigma” quantity in the exposure metadata does not imply that the returned DRP source catalog itself is simply a 5-sigma source list. These are different concepts. The catalog being reproduced correctly can still be the wrong catalog for a particular science benchmark if its design goal is reliability for calibration rather than completeness for source detection.

Different meanings of “detection” in the Rubin pipelines

This investigation highlights that there are multiple meanings of “detection” at different stages of the Rubin pipelines.

In the DRP single-frame path, the source catalog is calibration-oriented. It is not designed to be the definitive 5-sigma science source catalog for direct-image completeness studies.

In the AP path, the focus is different again. AP is primarily built around transients, variability, and difference-image detections. Its main outputs are the `dia_source_*` products measured on the difference image. These are crucial for alert production and transient science, but they are not the same thing as the direct-image source catalog needed for benchmarking a single-frame source detector.

This means that direct-image and difference-image detections should not be conflated. DRP and AP serve different purposes, and a source being “detected” in one context is not necessarily the same as being present in the other.

Final workflow

In the end, I split the workflow into image calibration and source detection. I use ISR followed by `CalibrateImageTask` only to produce the properly calibrated single-visit image and background, so the image itself matches the standard LSST pipeline processing. I then do not use the catalog returned by `CalibrateImageTask`, because that catalog is designed for calibration and is effectively more restrictive than the 5-sigma direct-image detector I want for my benchmark. Instead, I run a separate `SingleFrameDetectAndMeasureTask` on the calibrated image with an explicitly defined 5-sigma threshold. I do this because the goal of the benchmark is not to reproduce the default DRP calibration-source catalog, but to create a clean and interpretable single-visit 5-sigma source detector on top of the correctly processed LSST image.

Mar 8, 2026 | 📅 LSST Streak Detection Weekly

Single-frame source catalogs: three different answers for the same detector

After resolving the large LSSTComCam-specific offset in the earlier m5 study and moving the completeness experiment to LSSTCam, I encounter a second and more methodological problem: I realize that I am not actually reproducing the same single-frame detection product that exists in the Butler collections. This becomes obvious when I compare the number of detected sources for the same detector using different ways of running the pipeline tasks in Python.

At this stage, I find three substantially different source counts for what I initially think should be the same single-frame catalog:

- Catalog taken directly from Butler: about 640 sources
- Catalog produced with the old `CharacterizeImageTask.run()` followed by `CalibrateTask.run()` workflow: about 880 sources
- Catalog produced with the newer `CalibrateImageTask.run()` workflow: about 365 sources

This is a strong indication that I am not reproducing the same processing step in all three cases, and that my notebook-based simulation of the pipeline is not faithful to the actual pipeline products.

The first important discovery is that my previous Python implementation uses a deprecated task sequence. I am running:

1. `IsrTask`
2. `CharacterizeImageTask`
3. `CalibrateTask`

with the idea that this approximates single-frame source detection. However, once I begin varying the threshold, for example 5 sigma, 4 sigma, and 3 sigma, this approach produces

source counts and behavior that are clearly inconsistent with the actual LSSTCam pipeline outputs. This suggests that the older CharacterizeImageTask + CalibrateTask path is no longer the correct way to reproduce the single-frame products present in the current Butler collections.

The immediate need is therefore to understand how to run the modern CalibrateImageTask properly from within Python, without relying on piperun, and to determine exactly which task and configuration correspond to the single-frame catalog that I want to benchmark against.

Mar 1, 2026 | 📅 LSST Streak Detection Weekly

In the earlier completeness study, I injected point sources into single-visit images and compared their recovery to the reported single-visit limiting magnitude, m_5 , taken from `exp.info.getSummaryStats().getMagLim()`. The original expectation was that sources injected near this value should have roughly a 50% chance of being detected. However, the first version of this test showed a much larger offset than expected: the 50% completeness point appeared significantly brighter than the reported m_5 .

At first glance, this suggested that either the reported m_5 was inaccurate or that my injection-and-recovery framework was not measuring the same quantity as the LSST pipeline.

The original LSSTComCam result

The first version of the experiment was performed on LSSTComCam `preliminary_visit_image` data. In that setup, I found that injecting point sources at `mag = magLim` led to detection probabilities of only about 10–20%, and that the 50% completeness point was approximately 0.25–0.35 mag brighter than `getMagLim()`.

This was an important first result, but it later became clear that it should not be interpreted as a general statement about visit-level m_5 . The reason is that this first experiment used LSSTComCam data affected by a known bug in the m_5 calculation.

LSSTComCam-specific issue

A follow-up Slack discussion clarified that LSSTComCam had a bug related to ISR units / gain handling, which introduced an offset of about 0.28 mag in the reported m_5 values. This is essentially the same scale as the discrepancy I had measured in the original LSSTComCam injection experiment.

This meant that the large offset I initially reported was not purely a property of source detection or completeness. It was strongly influenced by a known problem in the LSSTComCam m_5 calculation itself.

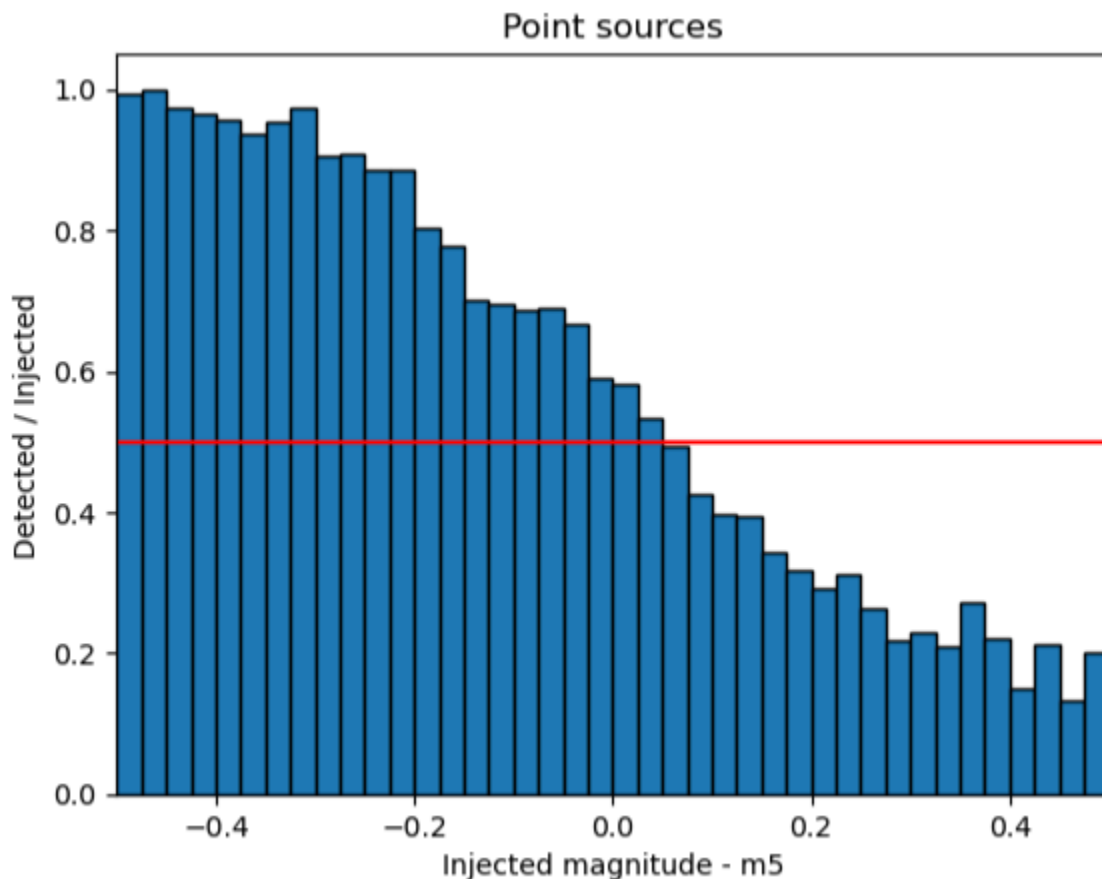
Switching to LSSTCam

To remove this ambiguity, I repeated the injection experiment on LSSTCam data, using reprocessed single-visit products from:

```
repo = dp2_prep  
collection = LSSTCam/runs/DRP/DP2/v30_0_0/DM-53881/stage2
```

I selected a random sample of detectors from August–September 2025 and repeated the controlled point-source injection test. In this LSSTCam sample, the behavior looked much more reasonable. The earlier large offset disappeared, and the 50% completeness point was much closer to the reported exposure-summary-statistics m_5 . The remaining offset was only of order ~ 0.07 mag, and completeness rose to nearly 100% within about 0.3 mag of m_5 .

This was consistent with the interpretation that the earlier LSSTComCam result had been dominated by the already-corrected bug.



Important conceptual clarification: three different quantities

At this stage it became clear that there are really three different quantities that need to be distinguished:

1. Exposure-summary-statistics m5: This is the value returned by `getMagLim()` / `stats_mag_lim`.
2. Catalog SNR = 5 magnitude: This is the magnitude at which objects in the source catalog have measured $\text{SNR} \approx 5$, or equivalently magnitude error ≈ 0.2 .
3. 50% completeness magnitude: This is the magnitude at which injected sources are recovered with 50% probability.

These three quantities are related, but they are not identical. The LSSTCam results suggested that the catalog SNR = 5 magnitude and the 50% completeness magnitude are closer to each other than either is to the exposure-summary-statistics m5. This is a more nuanced and physically reasonable picture than the original LSSTComCam result seemed to suggest. However this suggests that we can use SNR 5 as a 50% completeness proxy from now on on LSSTCam data.

Feb 9, 2026 | 📅 LSST Streak Detection Weekly

Two SNRs

An important aspect that I overlooked before is the distinction between two different signal-to-noise ratio (SNR) definitions that appear in the LSST pipelines and are often conflated.

The first is photometric (measurement) SNR, defined as:

$$\text{SNR}_{\text{phot}} = \frac{F_{\text{PSF}}}{\sigma_{F, \text{PSF}}}$$

Where F_{PSF} is the PSF-fit flux measured at a known source position, and $\sigma_{F, \text{PSF}}$ is its estimated uncertainty. This SNR is evaluated after detection and characterisation, assumes knowledge of the source location, and is the quantity reported in catalog columns `base_PsfFlux` / `base_PsfFluxErr`. It is the relevant SNR for photometric precision and scientific measurement, but it is not the quantity that controls whether a source is detected in the first place.

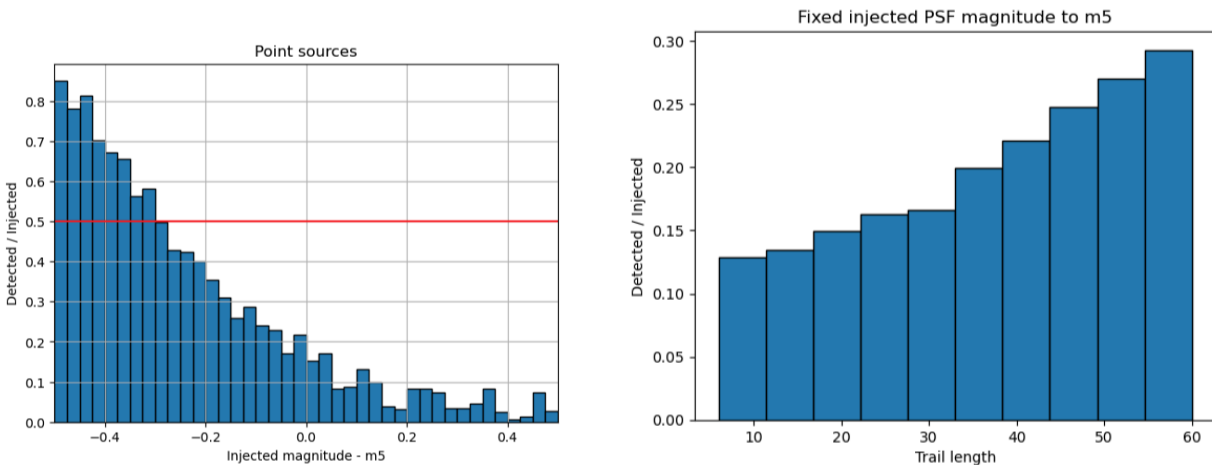
The second is detection SNR, which is implicitly used by the source detection algorithm. Detection operates on a PSF-convolved image and identifies peaks whose signal exceeds a threshold relative to the local noise estimate, nominally at the $\sim 5\sigma$ level. This SNR is defined with respect to the noise model used during detection (variance plane, gain, read noise, flat-fielding, etc.) and is closely related to the exposure-level limiting magnitude m5. Importantly, this SNR is evaluated without prior knowledge of the source position and depends on how signal is distributed spatially after convolution.

These two SNR definitions coincide only approximately for isolated point sources under ideal conditions. For trailed sources, they diverge significantly.

Limiting magnitude, completeness, and trail-length dependence

I performed controlled point-source and trailed-source injection experiments on LSSTComCam visit-level images to quantify how the reported limiting magnitude (m_5 , from `ExposureSummaryStats.getMagLim()`) relates to actual detection completeness. For point sources, injecting objects at m_5 does not yield a 50% detection probability: instead, completeness at m_5 is typically only ~10–30%, with the 50% completeness point occurring systematically at brighter magnitudes.

For trailed sources, completeness exhibits an additional, strong dependence on trail length even when the injected PSF magnitude is fixed to m_5 . Longer trails are detected with significantly higher probability than shorter trails at the same PSF magnitude. This behavior is physically expected, as longer trails deposit signal over a larger number of pixels above threshold after convolution, but it implies that detection completeness for moving objects cannot be described by a single magnitude limit alone.



Feb 2, 2026 | 📅 LSST Streak Detection Weekly

Empirical Stack SNR Correction

Despite the physically motivated model, validation against injected sources revealed a small but systematic residual bias between the predicted SNR and the Stack-measured SNR, primarily as a function of trail length. To correct for this, an empirical multiplicative correction factor

$$C(L)$$

was introduced, calibrated directly from injections with known stack detections. The final predicted SNR is given by

$$\text{SNR}_{\text{stack}} \approx \frac{F}{\sigma_F} \times C(L)$$

where $C(L)$ follows a smooth saturation-type function that approaches a constant value for long trails.

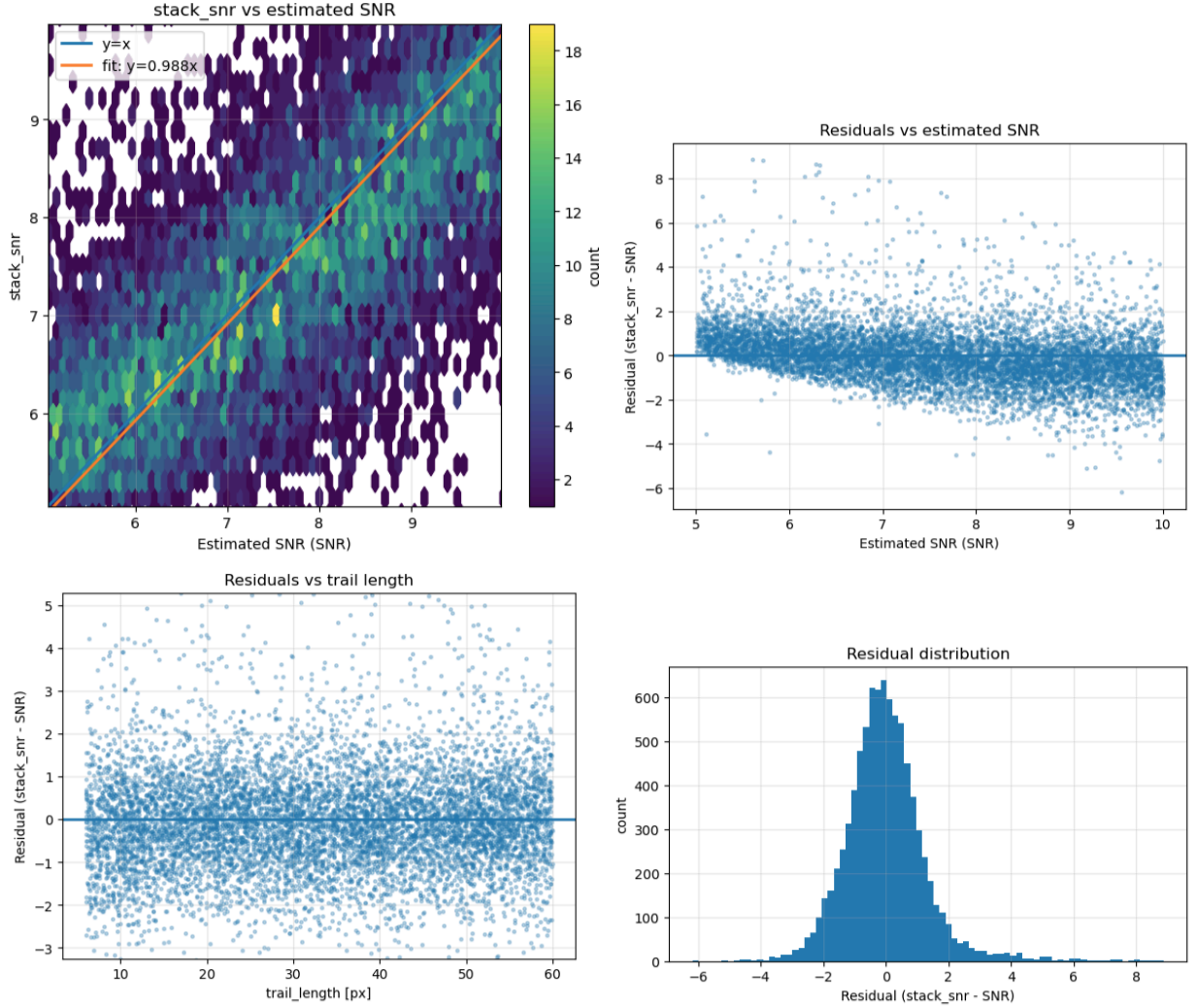
$$C(L) = a + b \cdot \frac{x}{1+x}$$

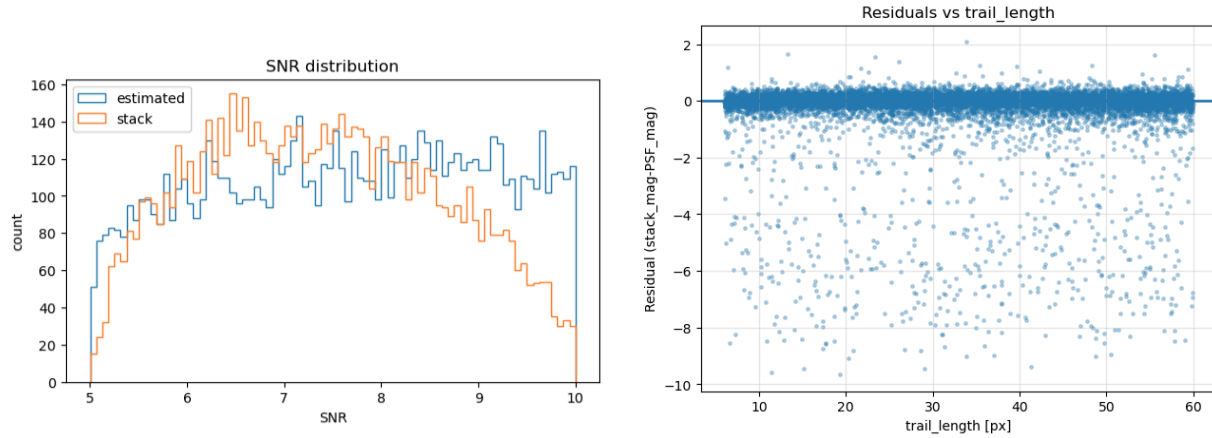
$$x = \left(\frac{L}{L_0} \right)^p$$

These are fitted values of parameters:

- $a = 1.12746468$
- $b = 0.16618154$
- $L_0 = 26.68594499$
- $p = 5.01963910$

This correction absorbs second-order effects not captured by the idealized mode. Importantly, the correction is applied only as a function of trail length and preserves the linear dependence of SNR on flux.





Jan 26, 2026 |  LSST Streak Detection Weekly

Estimation of Stack PSF-Flux Signal-to-Noise Ratio for Trailed Sources

Purpose

The purpose of this procedure is to estimate the signal-to-noise ratio (SNR) that would be obtained by an LSST Stack PSF-flux measurement when applied to a moving (trailed) source. Instead of explicitly running the Stack measurement tasks, the SNR is predicted analytically using the local PSF model, the pixel-level variance plane, and a physically motivated model of trail-induced signal loss. Throughout this document, the term “stack SNR” refers specifically to the expected SNR of a PSF-flux measurement (such as `base_PsfFlux`) applied to a trailed object.

Pixel and Noise Model

The pixel values in a small image stamp around the source are modeled as

$$d_i = F \cdot T_i + n_i$$

Where: i indexes pixels in the stamp, F is the true integrated instrumental flux of the source, T_i is the unit-flux trailed source template, n_i is noise with variance V_i .

The per-pixel variance V_i is taken directly from the exposure variance plane. Noise is assumed to be uncorrelated between pixels.

PSF-Flux Estimator

The LSST Stack PSF-flux measurement estimates flux by fitting a point-source PSF template to the pixel data using a weighted least-squares estimator. The estimated flux is

$$\hat{F}_{psf} = \frac{\sum_i (\phi_i \cdot d_i / V_i)}{\sum_i (\phi_i^2 / V_i)}$$

Where: ϕ_i is the PSF image evaluated at the source position, the PSF is normalized such that $\sum_i \phi_i = 1$. This estimator is optimal only when the true source morphology matches the PSF.

Trailed Source Morphology

A moving object does not appear as a point source during an exposure. Instead, its image is the convolution of the PSF with a straight line segment corresponding to its motion. The trailed template T is defined as the PSF convolved with a line of length L and direction θ , and is normalized such that

$$\sum_i T_i = 1$$

In the implementation, T is constructed numerically by summing many subpixel-shifted copies of the PSF along the trail direction and then renormalizing the result to unit integrated flux.

Expected Value of the PSF-Flux Measurement

Substituting the trailed source model into the PSF-flux estimator gives

$$\hat{F}_{psf} = \frac{\sum_i (\phi_i \cdot (F \cdot T_i + n_i) / V_i)}{\sum_i (\phi_i^2 / V_i)}$$

Taking the expectation value over noise yields

$$E[\hat{F}_{psf}] = \frac{F \cdot \sum_i (\phi_i \cdot T_i / V_i)}{\sum_i (\phi_i^2 / V_i)}$$

The ratio

$$m = \frac{\sum_i (\phi_i \cdot T_i / V_i)}{\sum_i (\phi_i^2 / V_i)}$$

quantifies the loss of signal due to the mismatch between the PSF template and the trailed source morphology. For a stationary point source, T equals ϕ and m equals 1. For trailed sources, m is less than 1 and decreases with increasing trail length.

Variance of the PSF-Flux Measurement

The variance of the PSF-flux estimator is determined solely by the noise and the PSF template and is given by

$$Var(\hat{F}_{psf}) = \frac{1}{\sum_i (\phi_i^2 / V_i)}$$

The corresponding standard deviation is

$$\sigma_{psf} = \frac{1}{\sqrt{\sum_i (\phi_i^2 / V_i)}}$$

This uncertainty does not depend on whether the source is trailed or stationary.

Definition of Stack SNR

The stack SNR is defined as the expected signal-to-noise ratio of the PSF-flux measurement:

$$SNR_{psf} = E[\hat{F}_{psf}] / \sigma_{psf}$$

Substituting the expressions above yields

$$SNR_{psf} = F \cdot \frac{\sum_i(\phi_i \cdot T_i / V_i)}{\sqrt{\sum_i(\phi_i^2 / V_i)}}$$

This equation describes how the measured SNR decreases as the overlap between the PSF template and the trailed source template decreases.

Effective Flux Uncertainty σ_F

For convenience, the SNR expression is rewritten in the form

$$SNR_{psf} = \frac{F}{\sigma_F},$$

where the effective flux uncertainty σ_F is defined as

$$\sigma_F = \frac{\sqrt{\sum_i(\phi_i^2 / V_i)}}{\sum_i(\phi_i \cdot T_i / V_i)}$$

This quantity incorporates both the noise properties of the image and the loss of sensitivity caused by PSF-trail mismatch.

The function `psf_fit_flux_sigma` computes σ_F by:

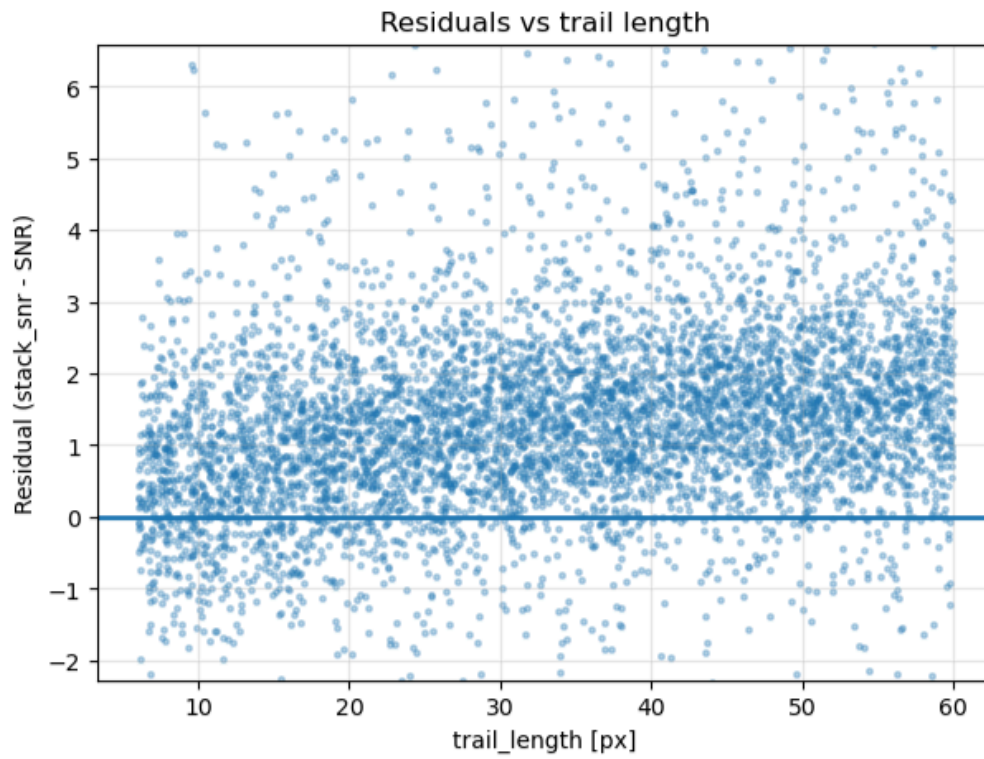
1. Evaluating the PSF at the source position and normalizing it to unit flux.
2. Choosing a sufficiently large image stamp to avoid truncation of PSF wings and trail extent.
3. Extracting the corresponding variance cutout from the exposure.
4. Constructing the trailed template T on the same stamp.
5. Computing the weighted sums $\sum(\phi^2 / V)$ and $\sum(\phi \cdot T / V)$.
6. Returning σ_F as defined above.

Conversion Between SNR and Magnitude

Once σ_F is known, the relationship between SNR and flux is linear:

$$F = SNR_{target} \cdot \sigma_F$$

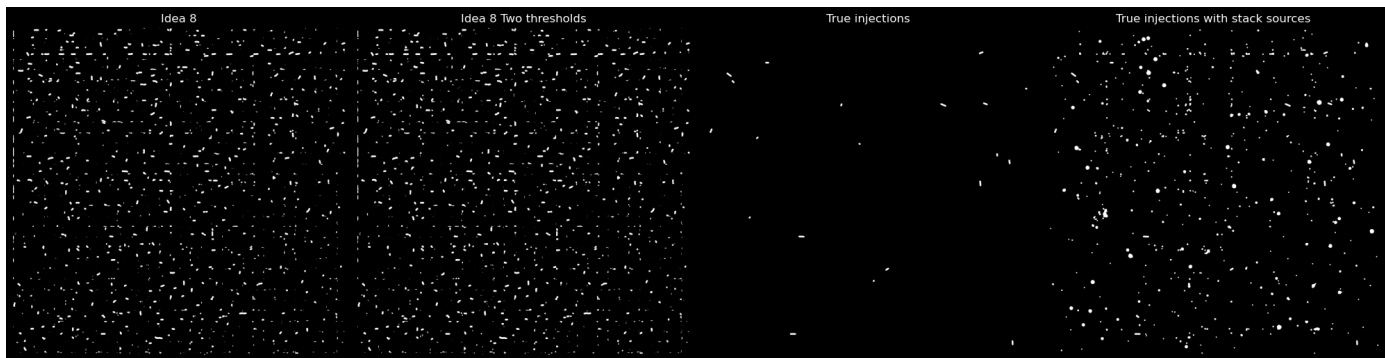
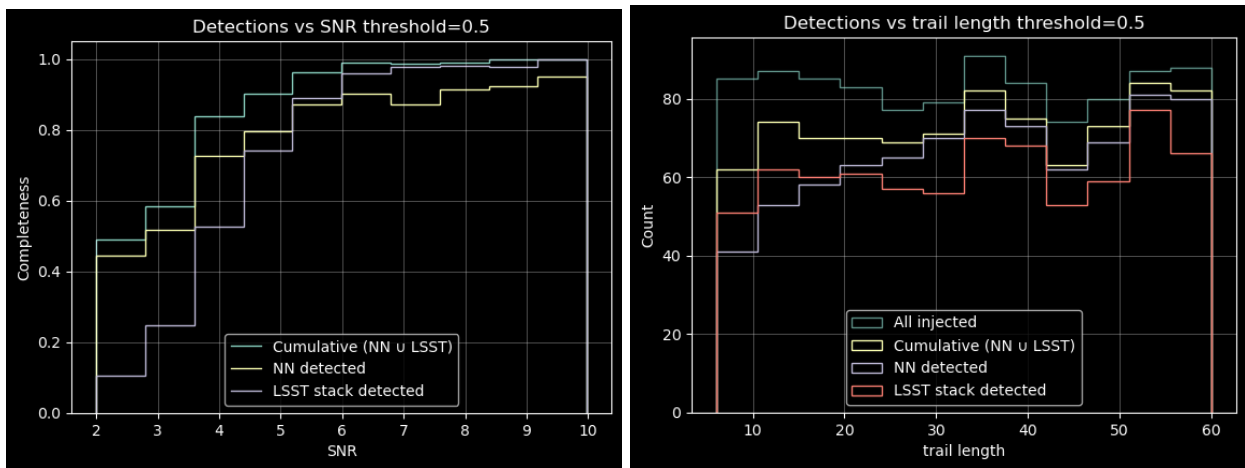
The instrumental flux F is then converted to an integrated magnitude using the exposure photometric calibration. This allows the code to sample a desired distribution of stack SNR values and convert them into physically consistent magnitudes for injected trailed sources.



LSST pipeline

Objectwise		
	Not Detected	Detected
Sources already in images		133679
Injected sources	260	740
	F1: 0.0109	F2: 0.0267

NN model



ML algorithm	Pixelwise		Objectwise	
Thr = 0.293	Not Detected	Detected	Not Detected	Detected
Places where nothing was injected (may have existing background sources)	792596293 pix	20797386 pix		51300 sources (note: these are only sources detected by the NN but not by the LSST stack)
Injected sources	396006 pixels	610315 pixels	212 sources	788 sources
	F1: 0.0545	F2: 0.1200	F1: 0.0297	F2: 0.0702

This week, I checked how I do SNR estimation. [Notebook](#)

SNR estimation

1) Random location, trail length and SNR

For each injection (loop index k), I draw a random pixel position:

```
x_pos = rng.uniform(1, dimensions.x - 1)
y_pos = rng.uniform(1, dimensions.y - 1)
```

Next, I convert that to on-sky coordinates:

```
sky_pos = raw.pixelToSky(x_pos, y_pos)
ra_pos = sky_pos.getRa().asDegrees()
dec_pos = sky_pos.getDec().asDegrees()
```

Trail length in pixels is then drawn:

```
inject_length = rng.uniform(*trail_length)
```

Lastly, I get SNR:

```
snr = rng.uniform(mag[0], upper_limit_mag)
```

2) Convert target SNR $\rightarrow$ PSF magnitude

I do:

```
psf_magnitude = snr_to_mag(snr, calexp, x_pos, y_pos)
```

What `snr_to_mag` does:

```
def snr_to_mag(snr, calexp, x, y):
    sigmaF = psf_fit_flux_sigma(calexp, x, y)
    F = snr * sigmaF
    mag = calexp.getPhotoCalib().instFluxToMagnitude(F)
    return mag
```

3) How `sigmaF` is computed

In `psf_fit_flux_sigma(calexp, x, y)`, I am computing the 1σ uncertainty of the best-fit PSF amplitude (flux). For data in pixel i : $d_i = Fp_i + n_i$, F is the (instrumental) flux I fit, p_i is the PSF template value in pixel i normalized to 1, and noise n_i is an independent Gaussian with variance v_i , taken from the exposure variance plane. The weighted least squares estimate of F is:

$$d_i = F p_i + n_i,$$

$$\hat{F} = \frac{\sum_i \frac{p_i d_i}{v_i}}{\sum_i \frac{p_i^2}{v_i}},$$

$$\sigma_F = \sqrt{\text{Var}(\hat{F})} = \left(\sum_i \frac{p_i^2}{v_i} \right)^{-1/2}$$

```
def psf_fit_flux_sigma(calexp, x, y):
    psf = calexp.getPsf()
    var_full = calexp.variance.array.astype(np.float64)

    # get the local PSF image evaluated at (x,y)
    p = psf.computeImage(geom.Point2D(x, y)).array.astype(np.float64)
    ph, pw = p.shape

    # center of stamp in full-image coords
    cx = int(round(x))
    cy = int(round(y))

    # stamp bounds in full image
    x0 = cx - pw//2
    y0 = cy - ph//2
    x1 = x0 + pw
    y1 = y0 + ph

    H, W = var_full.shape
    # clip to image bounds
    ix0 = max(x0, 0); iy0 = max(y0, 0)
    ix1 = min(x1, W); iy1 = min(y1, H)

    # corresponding bounds in PSF stamp coords
    px0 = ix0 - x0; py0 = iy0 - y0
    px1 = px0 + (ix1 - ix0); py1 = py0 + (iy1 - iy0)

    p_cut = p[py0:py1, px0:px1]
    # extract the corresponding stamp from the exposure variance plane
    v_cut = var_full[iy0:iy1, ix0:ix1]

    # normalize PSF cut to sum=1 (important if clipping happened)
    s = p_cut.sum()
    if not np.isfinite(s) or s <= 0:
```

```

    p = None
    v = None
else:
    p = p_cut / s
    v = v_cut
if p is None:
    return np.nan
good = np.isfinite(v) & (v > 0) & np.isfinite(p)
denom = np.sum((p[good]**2) / v[good])
if denom <= 0:
    return np.nan
return float(np.sqrt(1.0 / denom))

```

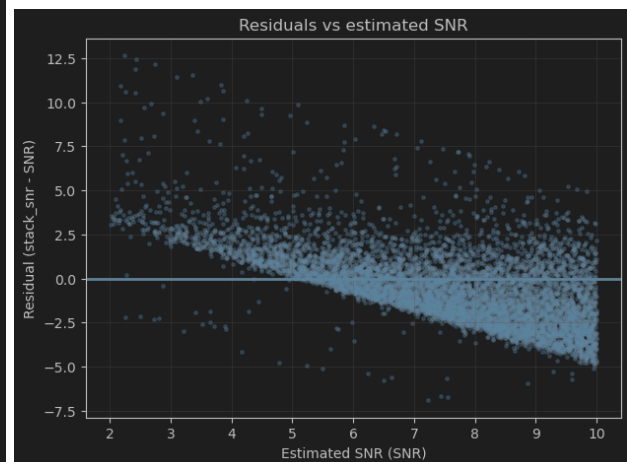
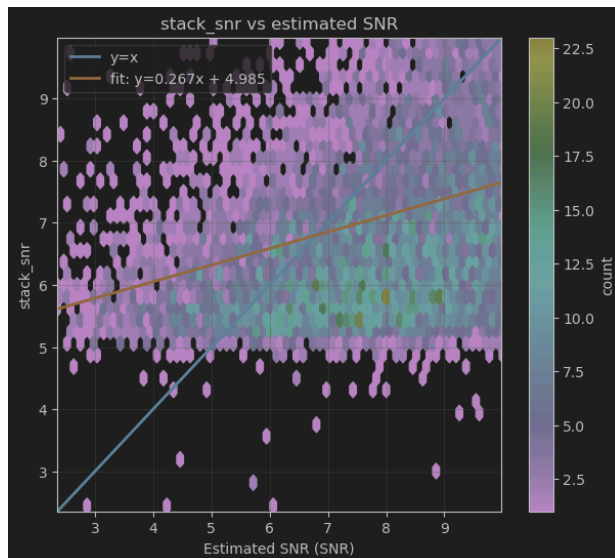
4) Convert PSF magnitude to integrated trail magnitude and surface brightness

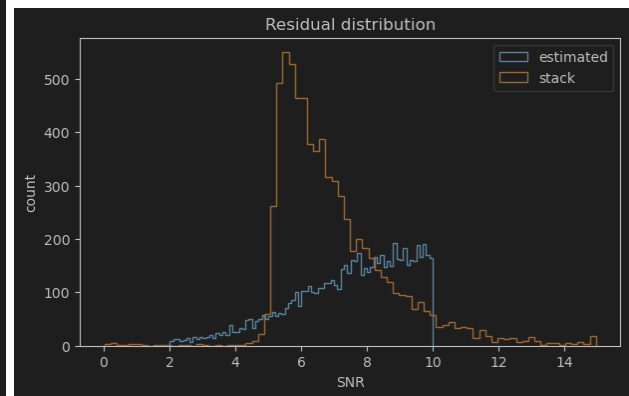
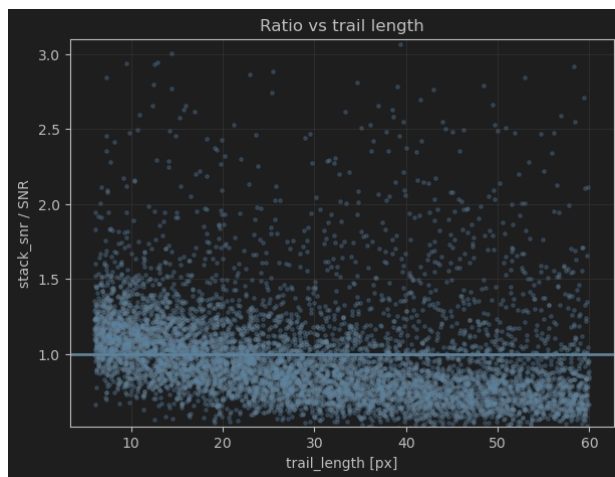
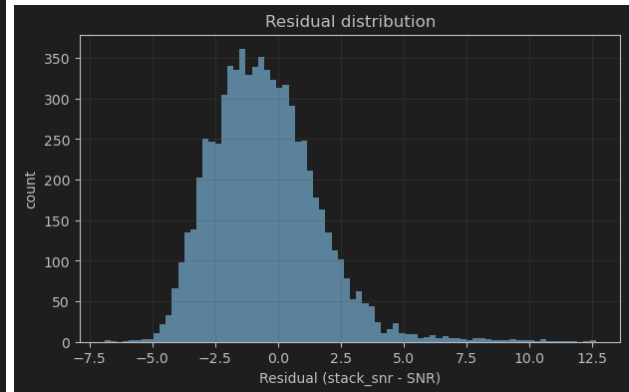
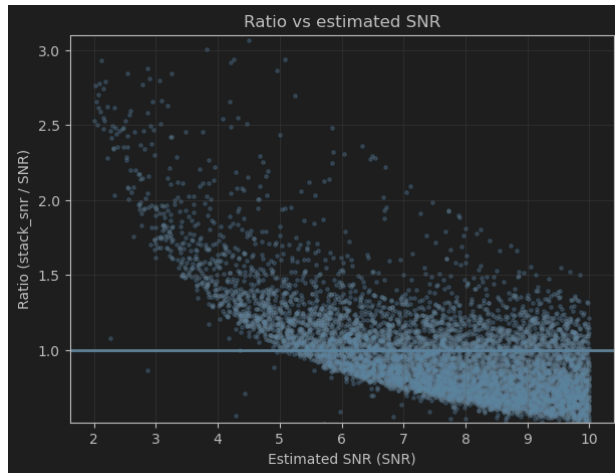
```

x = inject_length / (24*theta_p)
theta_p = fwhm(filter) * pixelScale
a=0.67, b=1.16
magnitude = psf_magnitude - 1.25*log10(1 + (a*x^2)/(1 + b*x))
surface_brightness = magnitude + 2.5*log10(inject_length)

```

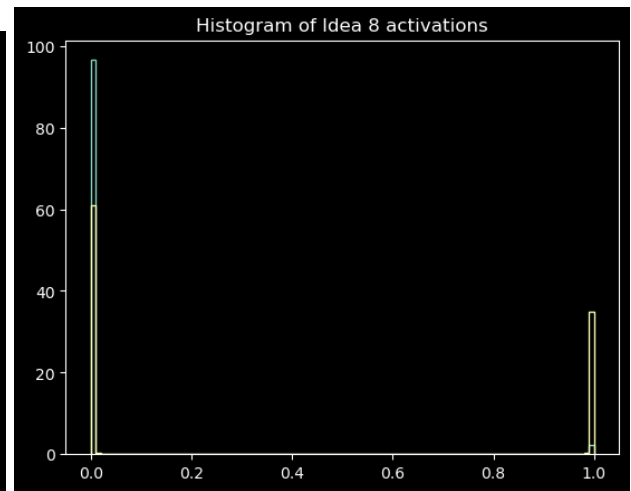
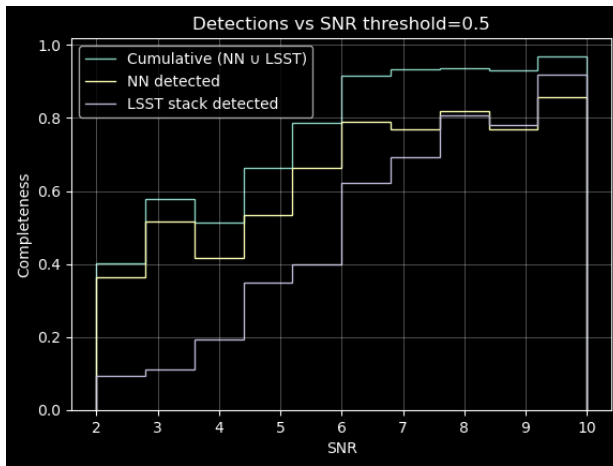
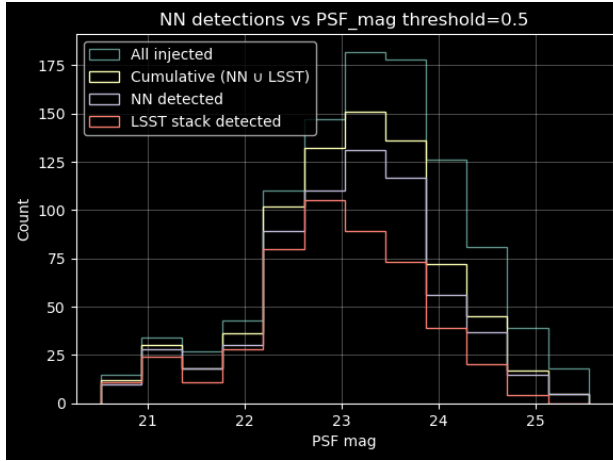
Results





Idea 8

Combine oversampling LSST missed streaks, with Focal twersky loss, and ignoring LSST detected (real) non-trail sources in loss with two-step thresholds.



ML algorithm	Pixelwise		Objectwise	
Thr = 0.293	Not Detected	Detected	Not Detected	Detected
Places where nothing was injected (may have existing background sources)	787966157 pix	25426449 pix		74806 sources (note: these are only sources detected by the NN but not by the LSST stack)
Injected sources	624648 pixels	382746 pixels	356 sources	644 sources
	F1: 0.0285	F2: 0.0641	F1: 0.0168	F2: 0.0405

Jan 12, 2026 | 📅 LSST Streak Detection Weekly

This week, I focused on improving the evaluation framework and on producing new training and testing datasets, which required retraining the model. [NOTEBOOK](#)

- I regenerated the train and test datasets with a uniform distribution in SNR and trail length (mag_length), instead of the previously used uniform distribution in SNR magnitude.
- Due to the change in the data distribution, the model was retrained on the new datasets.
- In addition, I created two specialized test datasets with a fixed trail length of 20 pixels, Fixed SNR = 5
Fixed SNR = 6
These datasets are intended for targeted performance studies close to the detection limit.

Evaluation improvements

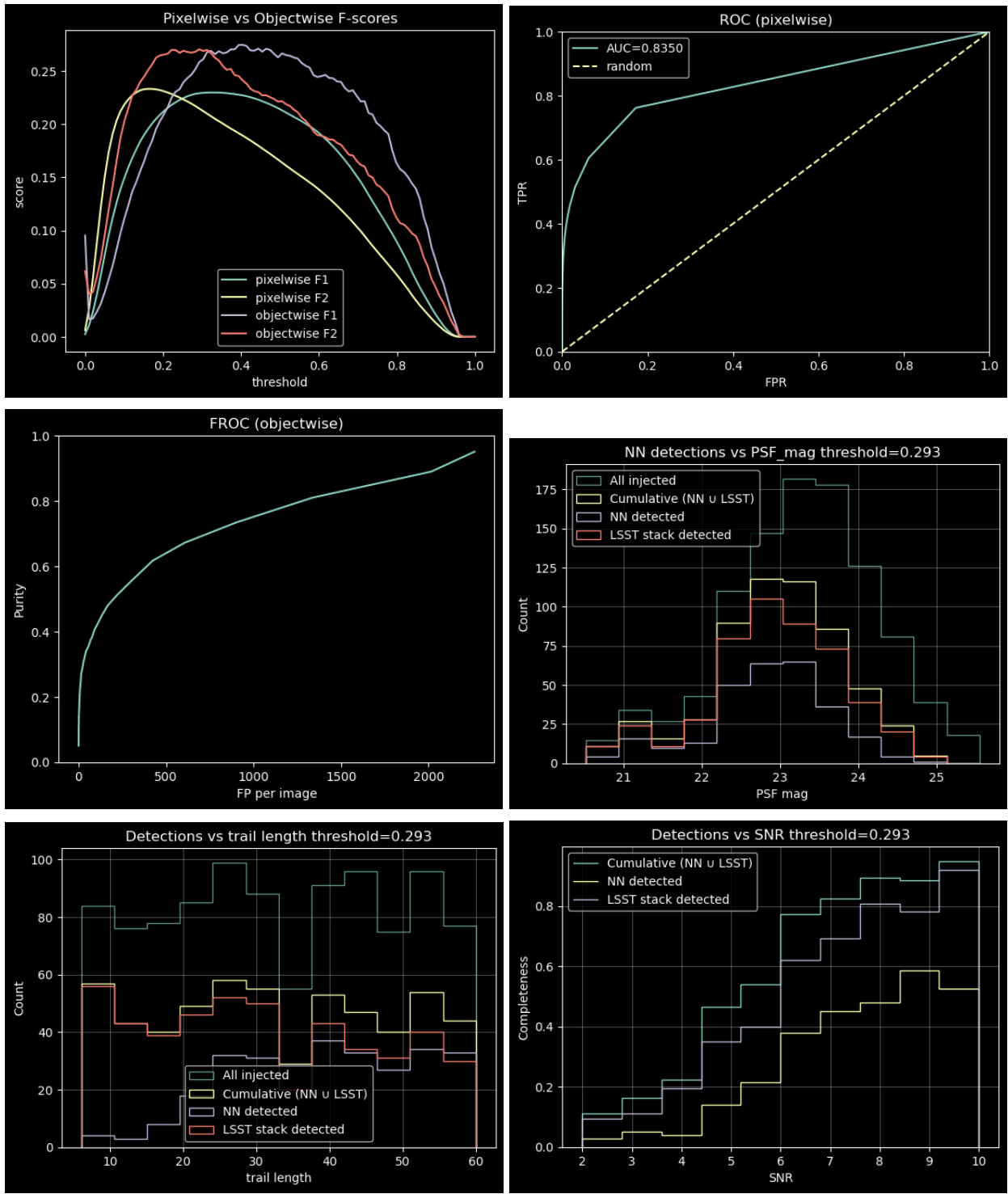
- I corrected the definition of false positives (FPs) in the evaluation. Previously, all unmatched detections were counted as FPs. Now, FPs are defined strictly as objects detected by the neural network that are not detected by the LSST stack, i.e., true hallucinations.
- While revisiting this logic, I identified a bug in the previous FP calculation, which caused FPs to be significantly underestimated in last week's results. This bug has now been fixed, and all recent evaluation results use the corrected FP definition and implementation.

Test set: SNR [2,10], Train len [6,60]

LSST pipeline

Objectwise		
	Not Detected	Detected
Sources already in images		133679
Injected sources	516	484
	F1: 0.0072	F2: 0.0175

Baseline Model



ML algorithm	Pixelwise		Objectwise	
Thr = 0.293	Not Detected	Detected	Not Detected	Detected

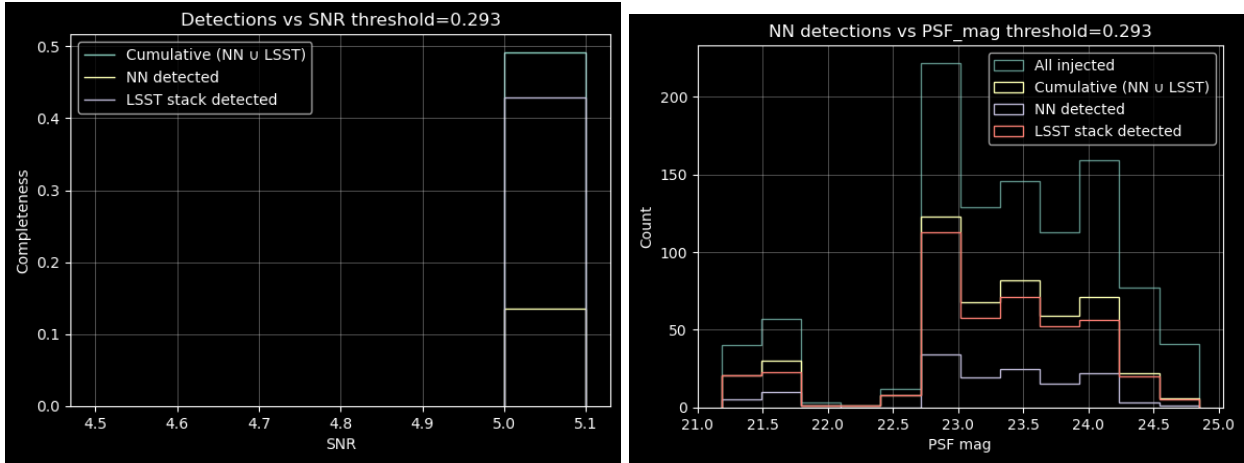
Places where nothing was injected (may have existing background sources)	812795451 pix	597155 pix		854 sources (note: these are only sources detected by the NN but not by the LSST stack)
Injected sources	799982 pixels	207412 pixels	721 sources	279 sources
	F1: 0.2289	F2: 0.2145	F1: 0.2616	F2: 0.2718

Test set: SNR 5, Train len 20

LSST pipeline

Objectwise		
Thr = 0.5 sigma	Predicted Negative	Predicted Positive
Actual Negative		133679
Actual Positive	571	429
	F1: 0.0064	F2: 0.0155

Baseline Model



	Pixelwise	Objectwise
--	-----------	------------

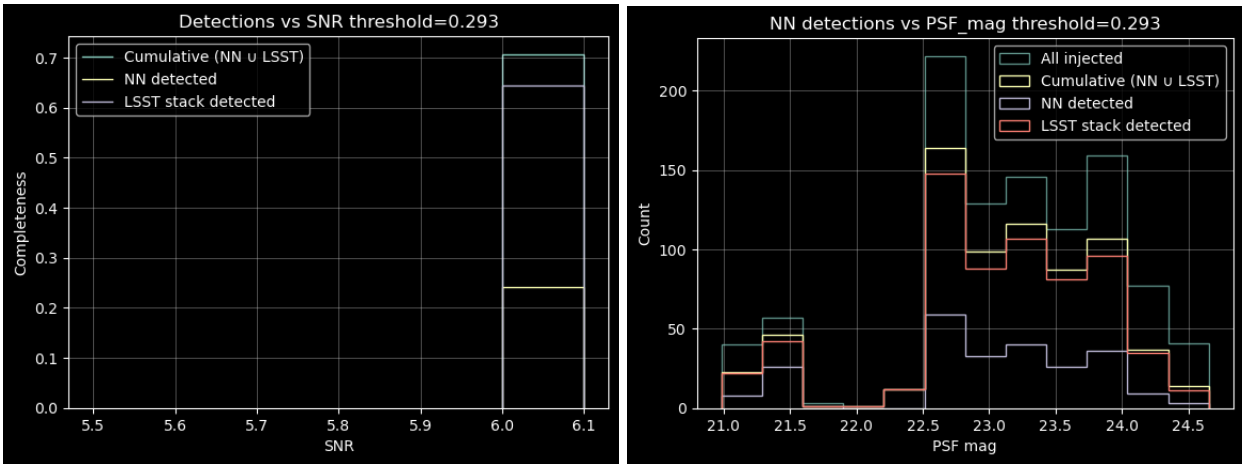
Thr = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive
Actual Negative	813092009	578328		851
Actual Positive	676360	53303	866	134
	F1: 0.0783	F2: 0.0751	F1: 0.1350	F2: 0.1344

Test set: SNR 6, Train len 20

LSST pipeline

Objectwise		
	Predicted Negative	Predicted Positive
Actual Negative		133679
Actual Positive	356	644
	F1: 0.0095	F2: 0.0233

Baseline Model



	Pixelwise		Objectwise	
Thr = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive

Actual Negative	813083269	587068		854
Actual Positive	630930	98733	760	240
	F1: 0.1395	F2: 0.1370	F1: 0.2292	F2: 0.2356

Jan 5, 2026 | 📅 LSST Streak Detection Weekly

I focus on improving the model, both purity and completeness, and I am going through the list of ideas I compiled last time. After a second thought, I rearranged the list order a little bit.

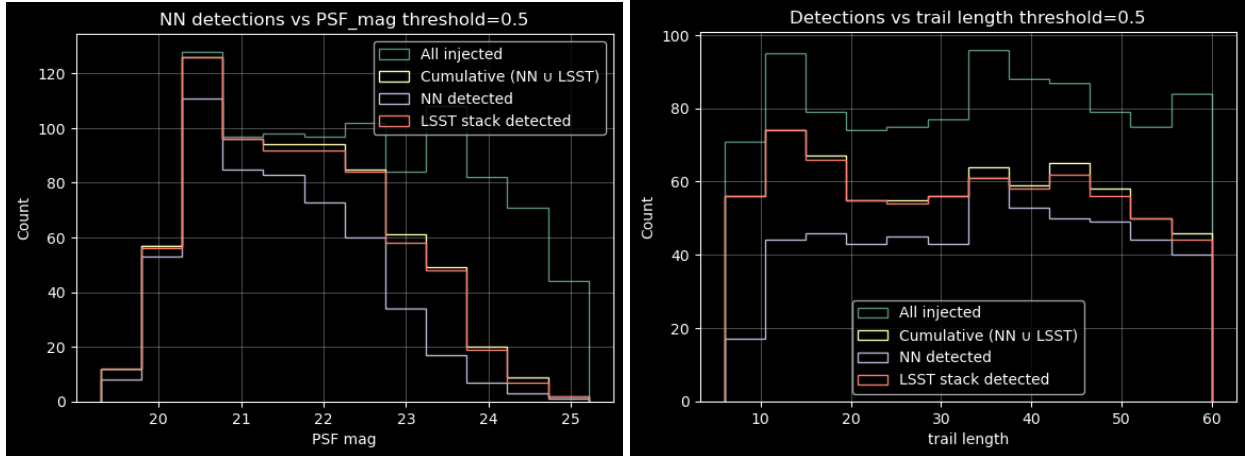
[Notebook](#)

LSST pipeline

Objectwise		
	Predicted Negative	Predicted Positive
Actual Negative		86588
Actual Positive	288	692
	F1: 0.0157	F2: 0.0379

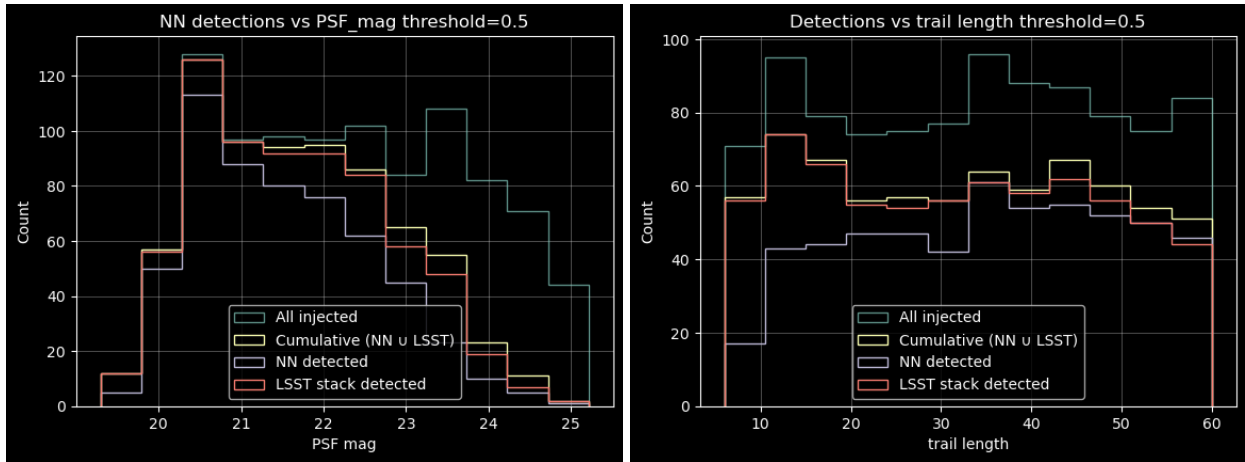
Baseline Model

	Pixelwise		Objectwise	
Thr = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive
Actual Negative	812612719	799866		139
Actual Positive	516949	470466	445	535
	F1: 0.4168	F2: 0.4506	F1: 0.6469	F2: 0.5823



Idea 1: Curriculum training biased to LSST-missed injections

Precompute a boolean label per injected object or crop: `lsst_detected` in $\{0,1\}$. Early training: sample from the full dataset normally. Late training: progressively biased sampling toward `lsst_detected = 0` samples. Evaluation remains on an unbiased test set, with separate tracking of LSST-missed and NN-only detections.

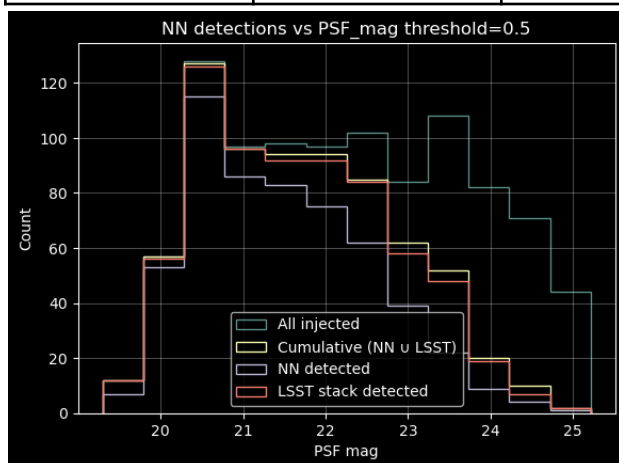


	Pixelwise		Objectwise	
Thr = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive
Actual Negative	812141440	1271145		415
Actual Positive	519115	468300	422	558
	F1: 0.3435	F2: 0.4116	F1: 0.5714	F2: 0.5702

Idea 2: Loss biased toward LSST-missed examples

Loss biased toward LSST-missed examples. Weight the loss contribution per sample using `lsst_detected`: higher weight for LSST-missed, lower for LSST-detected. Optionally apply this weighting only in later epochs.

	Pixelwise		Objectwise	
Thr = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive
Actual Negative	812318757	1093828		270
Actual Positive	504782	482633	424	556
	F1: 0.3765	F2: 0.4367	F1: 0.6157	F2: 0.5858

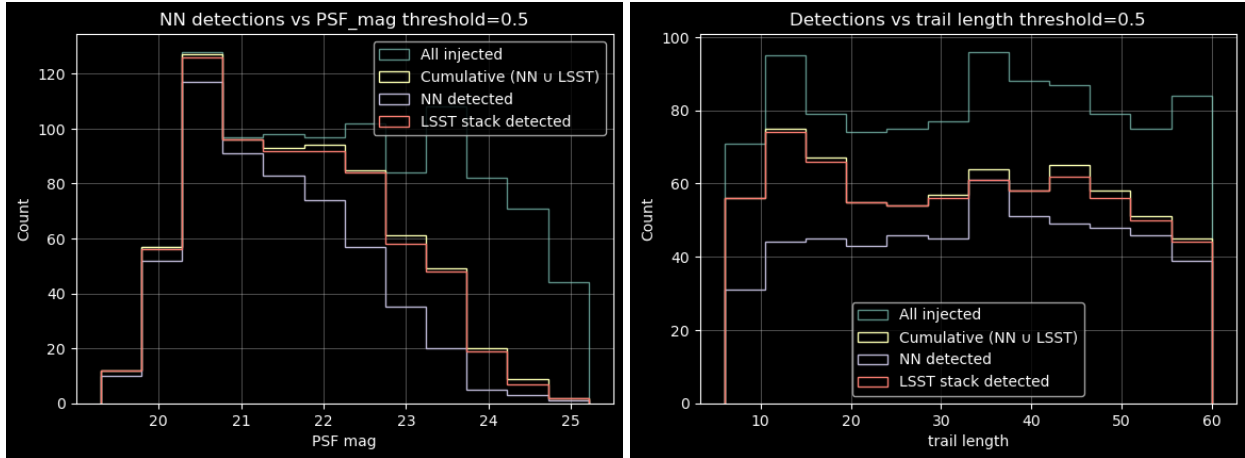


Idea 3: Two-phase loss schedule (BCE → Focal Tversky)

Phase 1: train with BCE (or BCE+Dice) for stability.

Phase 2: gradually blend in Focal Tversky: $L = (1 - \lambda) * L_{\text{BCE}} + \lambda * L_{\text{FocalTversky}}$, ramp λ from 0 to 1 over selected epochs. Reduce learning rate at the start of Phase 2.

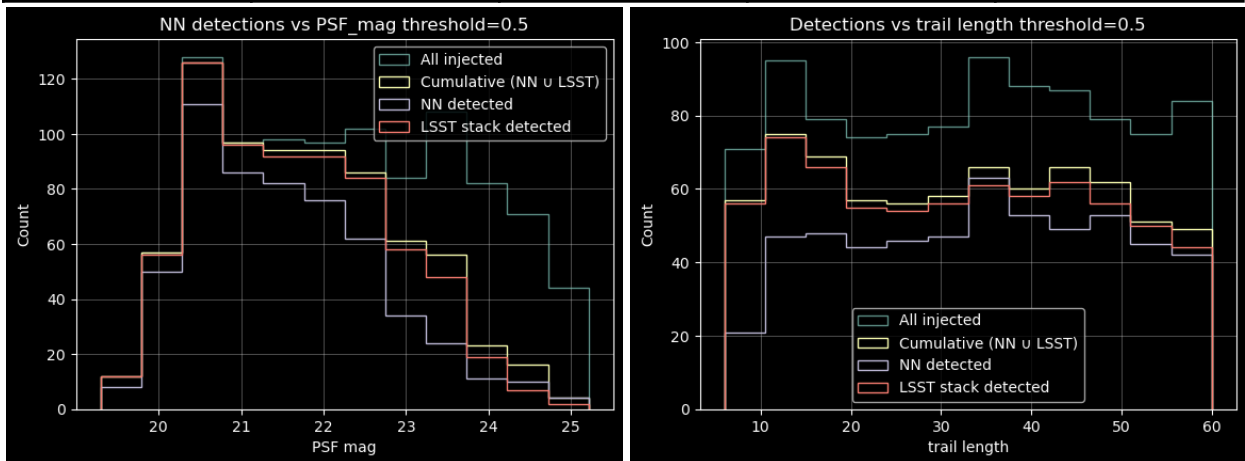
	Pixelwise		Objectwise	
Thr = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive
Actual Negative	812771760	640825		500
Actual Positive	522922	464493	432	548
	F1: 0.4439	F2: 0.4594	F1: 0.5404	F2: 0.5515



Idea 4: Two-stage detection: geometry + score

Keep NN output as a continuous probability map $p(y,x)$. Use a low pixel threshold t_{low} only to define connectivity: $\text{binary} = (p \geq t_{\text{low}})$. Run connected-component labeling with `pixel_gap` to obtain candidate regions. For each candidate, compute a score from original probabilities: e.g., $\max(p)$ or $\text{mean}(\text{top-}k \text{ } p)$. An injected trail is detected if: (a) a candidate overlaps the truth mask, and (b) its score $\geq \text{score_thr}$. t_{low} controls geometry/connectivity, score_thr controls detection confidence.

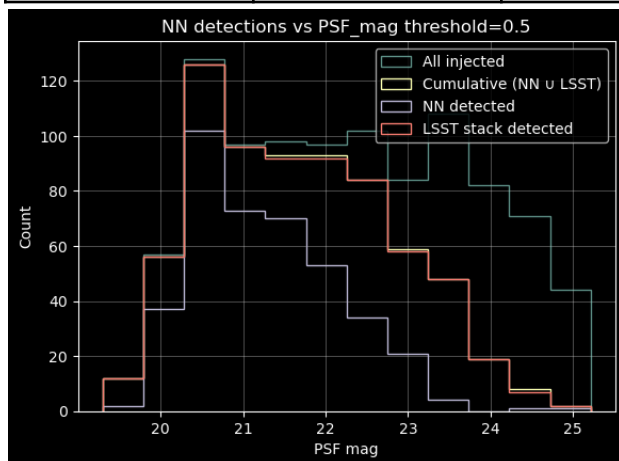
	Pixelwise		Objectwise	
Thr_low = 0.01 Thr_score = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive
Actual Negative	782343872	31068713		90
Actual Positive	408703	578712	422	558
	F1: 0.0355	F2: 0.0813	F1: 0.6855	F2: 0.6108



Idea 5: Line coherence bias (loss)

Encourage elongated, coherent trail-like predictions. Loss-based option: penalize isolated activations and compact blobs, encourage connectivity and elongation.

	Pixelwise		Objectwise	
Thr = 0.5	Predicted Negative	Predicted Positive	Predicted Negative	Predicted Positive
Actual Negative	813344558	68027		17
Actual Positive	693415	294000	582	398
	F1: 0.4357	F2: 0.3409	F1:0.5706	F2: 0.4591



Dec 22, 2025 | 📅 LSST Streak Detection Weekly

I focused on a systematic evaluation of the current neural-network streak detector, comparing pixel-level behavior, object-level behavior, and end-to-end detection performance against the LSST stack on injected data.

LSST stack results for 50 detectors

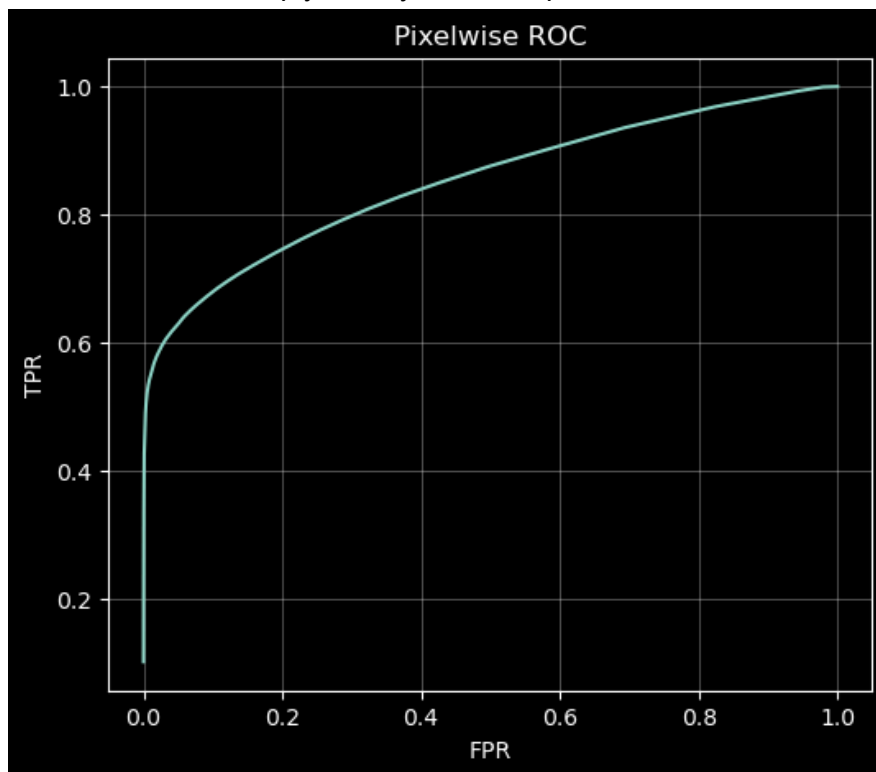
- True Positive: 692
- False Positive: 86588
- False Negative: 288
- Purity: 0.0079
- completeness 0.7061
- F1 score: 0.0157
- F2 score: 0.0379

I noticed that there are 86588 false positives by the LSST stack in the 50-detector test dataset. NN has a significantly lower FP number, depending on the threshold, from a few thousand down to a few hundred.

1. Pixel-level performance

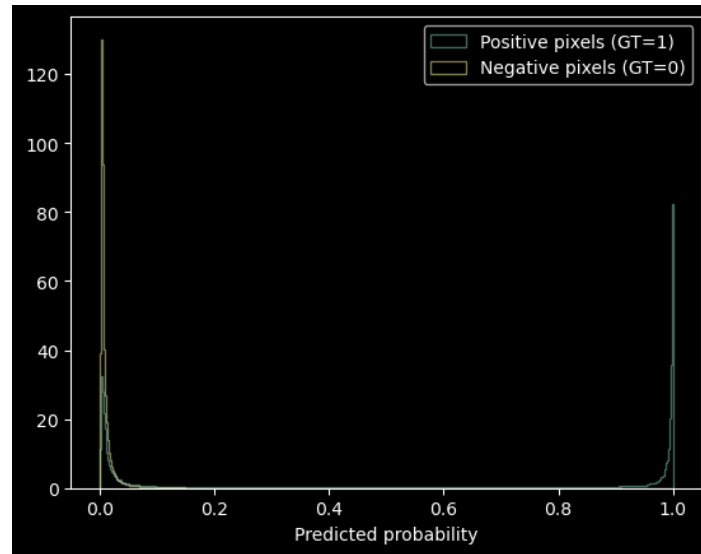
Pixelwise ROC

- The pixelwise ROC curve shows a strong separation between signal and background pixels.
- True-positive rate rises steeply at very low false-positive rates.



Probability distributions

- Negative (background) pixels are sharply peaked near 0.
- Positive (trail) pixels are strongly peaked near 1, but there is some overlap, possibly from the edges of the trails.

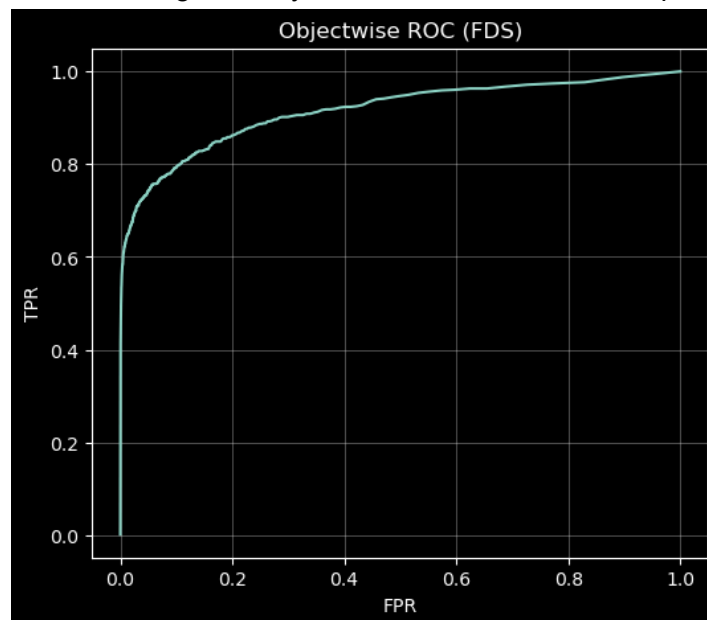


The probability map could provide enough information to support more sophisticated detection logic.

2. Object-level performance (FDS candidates)

Objectwise ROC (FDS)

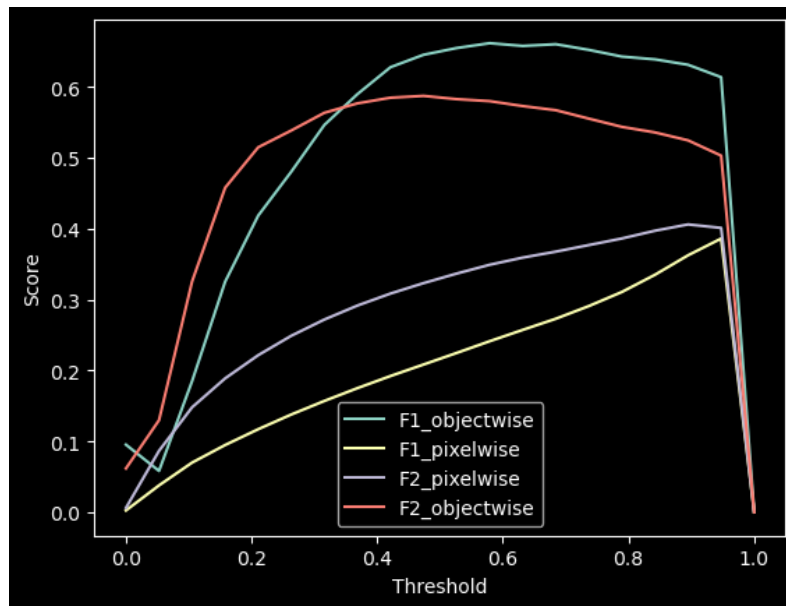
- Candidates are defined using a threshold and connected components (pixel-gap based grouping). The algorithm is a first-depth search.
- Candidate scores are derived from the original NN probabilities (e.g. max probability in the component).
- The object-wise ROC is significantly better behaved than naive pixel aggregation.



3. Threshold scans and F-scores

F1 / F2 vs threshold

- Pixelwise F1 and F2 peak at relatively high thresholds but saturate early.
- Objectwise F1 and especially F2 achieve higher values and are more stable across a broad threshold range.
- F2 favors recall, which is desirable for moving-object detection.



4. LSST comparison

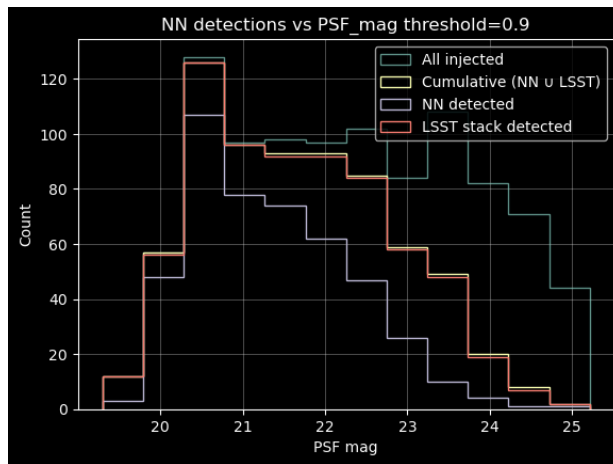
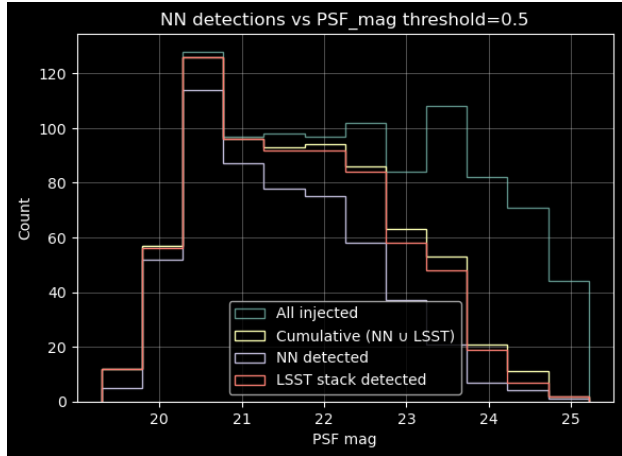
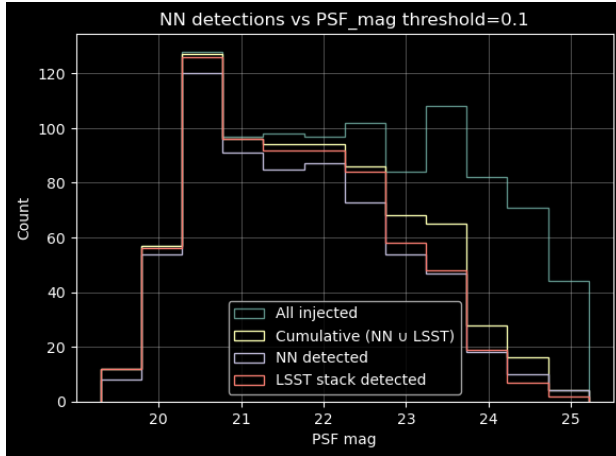
Detection completeness vs magnitude

Detection histograms were produced for three NN thresholds (0.1, 0.5, 0.9).

Results

- At low thresholds (≈ 0.1), NN detections are competitive with or exceed LSST stack detections at faint magnitudes.
- At higher thresholds, NN detections drop faster than LSST, as expected.
- The union ($NN \cup LSST$) consistently recovers more objects than either method alone.

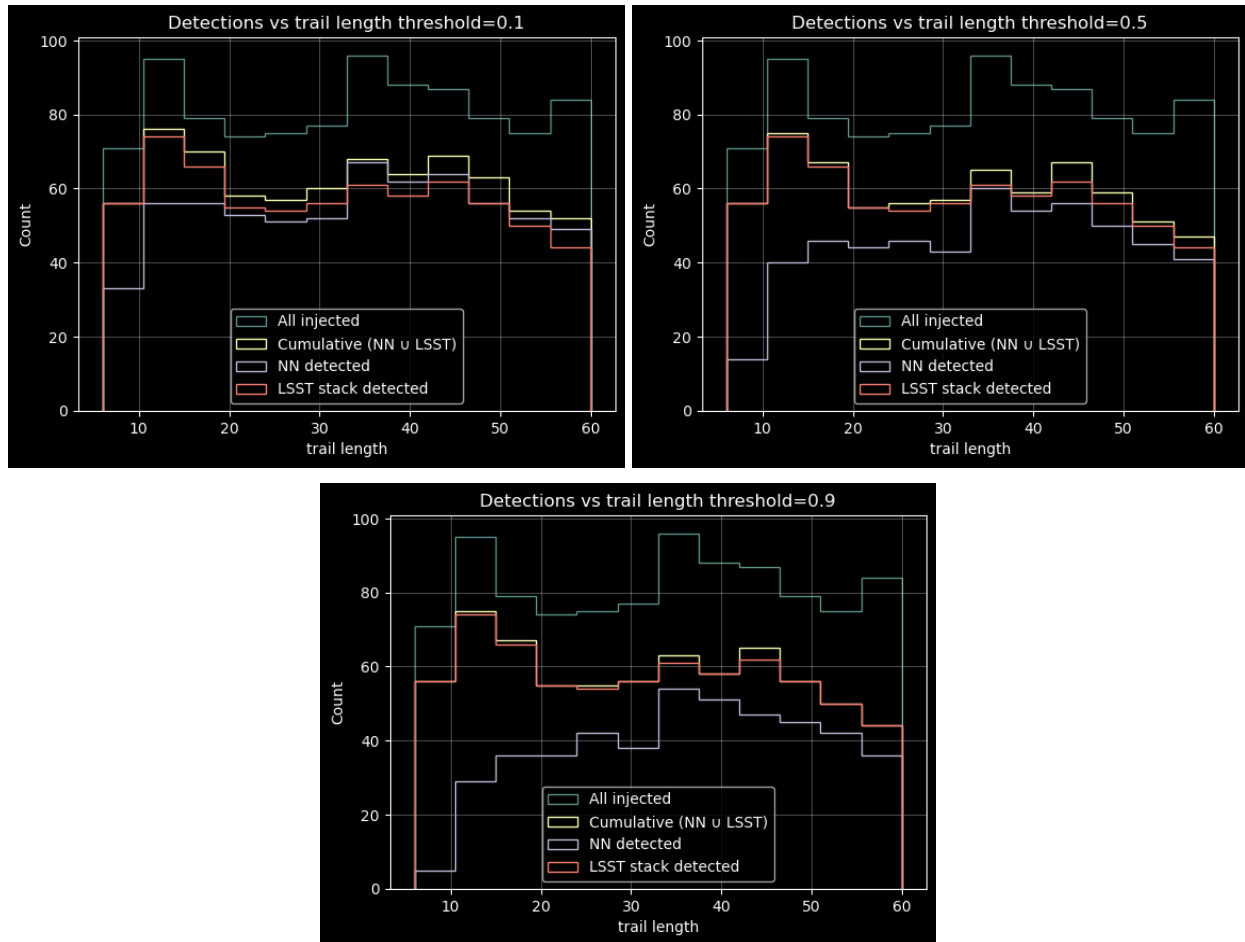
The choice of the threshold is not a trivial thing to do, since the decision about the number of allowed false positives is crucial. This needs more discussion! LSST stack has 86588 false positives, and NN is nowhere near that number.



5. Detection completeness vs trail length

- NN performance is relatively flat across trail lengths once above the very shortest trails.
- LSST stack detections degrade more strongly for shorter trails.
- The NN is less sensitive to trail length than the classical pipeline, especially at low thresholds.

This is not expected behaviour; we want to detect trails that LSST does not detect. Further refinement of the model is needed.



Main conclusion:

NN creates a comparable number of detections but with far fewer false positives (sometimes by 3 orders of magnitude less). Comparing NN and LSST on just the number of true positives is not fair, since the amount of false positives by the LSST stack is huge. If we choose a very low threshold for NN, we could easily beat it. In the trivial scenario, the threshold could be 0, and every pixel could be detected as the asteroid, which would push completeness to 100%. We need to define what the point of this algorithm is. Maybe the point is to create an algorithm that reliably detects asteroids, not just detects everything. The number of LSST false positives is very large; no need to add more false positives with NN. The problem definition is then the following: can we create an ML algorithm that does not significantly add any more false positives, while boosting the cumulative detection of LSST+NN detection?

Next steps:

Model evaluation

- ☐ Use a list of known objects that Jake Kurlander ran an association against SORCHA to see if we can recover known asteroid trails captured in the LSST Cam. Make sure to use objects faster than 1deg/day
- ☐ Create a ROC curve for LSST stack detection as well, where I would change the sigma detection threshold. This would be a fair comparison, since the number of false positives on the LSST stack is huge.

Refine model

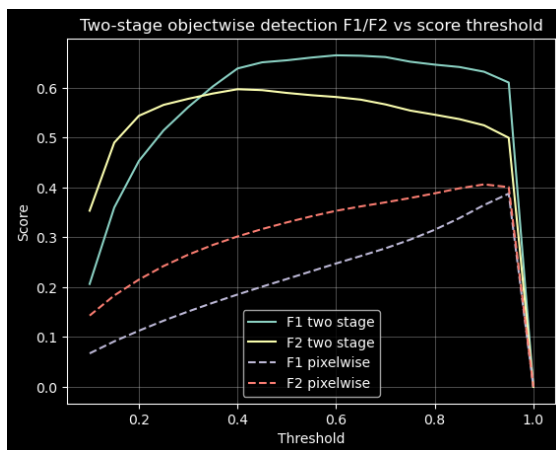
HIGH PRIORITY

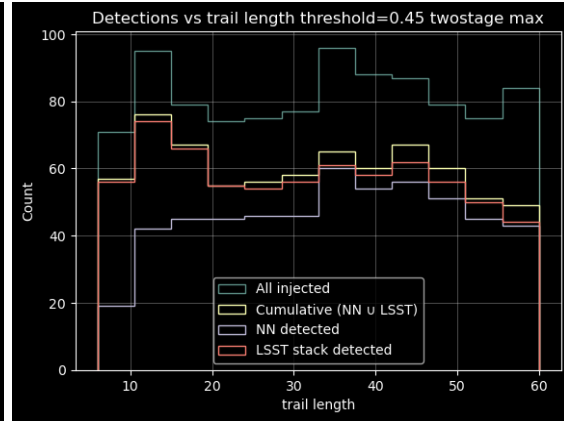
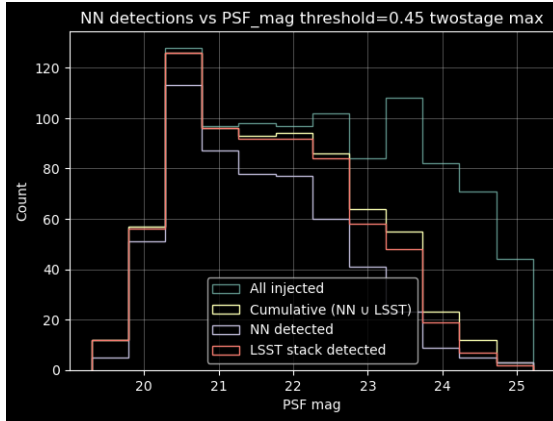
- ☒ Idea 1: **Two-stage detection: geometry + score**

Keep NN output as a continuous probability map $p(y,x)$. Use a low pixel threshold t_{low} only to define connectivity: $\text{binary} = (p \geq t_{\text{low}})$. Run connected-component labeling with `pixel_gap` to obtain candidate regions. For each candidate, compute a score from original probabilities: e.g., $\max(p)$ or $\text{mean}(\text{top-}k\ p)$. An injected trail is detected if: (a) a candidate overlaps the truth mask, and (b) its score $\geq \text{score_thr}$. t_{low} controls geometry/connectivity, score_thr controls detection confidence.

RESULT:

Slight improvement in detection at the best threshold, about 3% increase in detection compared to the LSST stack alone.





- Total injected: 980
- LSST detected: 692 70.61224489795919 %
- NN detected: 552 56.326530612244895 %
- Both detected: 721 73.57142857142857 %

MEDIUM PRIORITY

☐ Idea 2: **Candidate-level aggregation instead of pixel-level aggregation**

After candidate extraction, compute multiple features per candidate: max(p), top-k mean(p), percentile(p), pixel count/length, elongation (PCA eigenvalue ratio), bounding box aspect ratio, line-fit residuals, or orientation stability. Train a simple second-stage classifier (logistic regression or small MLP) using candidate features only. Use the classifier output as the candidate score for thresholding.

☒ Idea 3: **Curriculum training biased to LSST-missed injections**

Precompute a boolean label per injected object or crop: lsst_detected in {0,1}. Early training: sample from the full dataset normally. Late training: progressively biased sampling toward lsst_detected = 0 samples. Evaluation remains on an unbiased test set, with separate tracking of LSST-missed and NN-only detections.

☒ Idea 4: **Loss biased toward LSST-missed examples**

Weight the loss contribution per sample using lsst_detected: higher weight for LSST-missed, lower for LSST-detected. Optionally apply this weighting only in later epochs.

☒ Idea 5: **Line coherence bias (loss or auxiliary head)**

Encourage elongated, coherent trail-like predictions.

Loss-based option: penalize isolated activations and compact blobs, encourage connectivity and elongation.

Auxiliary-head option: add a head predicting local trail orientation or direction, trained from injection geometry or skeletonized truth masks.

☒ Idea 6: **Two-phase loss schedule (BCE → Focal Tversky)**

Phase 1: train with BCE (optionally BCE+Dice) for stability.

Phase 2: gradually blend in Focal Tversky: $L = (1 - \lambda) * L_{\text{BCE}} + \lambda * L_{\text{FocalTversky}}$, ramp λ from 0 to 1 over selected epochs. Optionally reduce learning rate at the start of Phase 2.

☐ Idea 7: **Explicit calibration of probability scale (post-training)**

Perform calibration on a validation set. Prefer candidate-level calibration: extract candidates, label them as true/false. Fit Platt scaling or isotonic regression to map raw candidate scores to calibrated scores. Use calibrated scores for setting `score_thr` consistently.

☐ Idea 8: **Postprocessing with LSD or Hough transform**

Apply classical line detection (e.g., Hough Transform, LSD) to the U-Net output to extract and validate linear streak candidates. May increase robustness and interpretability of detections.

LOW PRIORITY

☐ Idea 9: **Soft / distance-weighted ground truth**

Replace binary trail masks with soft targets peaked on the trail core. Construct targets using distance-to-centerline or PSF-like convolution. Train the network to regress these soft targets. Requires regenerating training targets.

☐ Idea 10: **Multi-scale supervision**

Add a coarse, lower-resolution output head in addition to the fine mask. Apply loss to both fine and coarse outputs: $L = L_{\text{fine}} + \mu * L_{\text{coarse}}$. Optionally fuse outputs at inference or keep the coarse head for training only.

Jul 24, 2024 - Dec 21, 2025 | 📅 LSST Streak Detection Weekly

During this period, the primary focus shifted from proof-of-concept development on HSC data to making the streak-detection pipeline operational and scientifically meaningful on LSST commissioning data, specifically LSSTComCam. With the start of Rubin commissioning, it became clear that any further progress required retraining and reevaluating the model directly on LSST-like data, rather than continuing to rely on HSC as a proxy. Achieving this turned out to be substantially more complex than anticipated and required a deep dive into the LSST pipeline, data model, calibration products, and source injection machinery.

A significant fraction of the effort went into understanding how to reliably generate a training dataset from LSSTComCam images, rather than HSC. This involved learning how ComCam data are structured in the Butler, how calibrations differ from HSC, how WCS, PSF, PhotoCalib, and exposure metadata are accessed, and how source injection behaves in this environment. In parallel, a large amount of work was dedicated to clarifying how different magnitude definitions (integrated magnitude, surface brightness, PSF magnitude) propagate through the injection pipeline and how they relate to LSST detection limits. This investigation uncovered several non-obvious details in `source_injection`, including how injected flux is internally rescaled by trail length and how this maps back to PSF magnitudes. Only after resolving these issues was it possible to generate injections with physically consistent brightness distributions and meaningful detection limits. While this work was largely infrastructural and not immediately visible in model metrics, it was essential for producing a trustworthy training and evaluation dataset.

In parallel with the dataset work, the entire machine-learning stack was reworked. The original model, implemented in Keras/TensorFlow, was retired and replaced with a fully PyTorch-based implementation. This decision was motivated by multiple factors: PyTorch is already used within parts of the LSST ecosystem, it integrates more naturally with existing analysis tools, and TensorFlow had become increasingly fragile and difficult to maintain in the LSST software environment. The transition was not a simple port: the full data-loading pipeline, preprocessing, tiling strategy, normalization, augmentation, and bookkeeping were redesigned to be consistent, reproducible, and efficient in PyTorch.

The model architecture itself was also substantially revised. While the core idea remains a U-Net–style fully convolutional network operating on image tiles with pixel-level outputs, the current model includes several major additions relative to the original implementation: attention mechanisms (including channel/spatial attention and attention gates on skip connections), deeper encoder–decoder paths, improved feature aggregation across scales, and more careful handling of class imbalance. These changes were introduced incrementally and tested empirically, with the goal of improving sensitivity to faint, elongated, low–low-surface-brightness streaks without inflating false positives from noise or artifacts. The architecture should now be viewed as a descendant of the original U-Net, rather than a minor modification.

Loss-function design and training strategy also evolved significantly. Early experiments confirmed that naive losses perform poorly due to the extreme pixel-level class imbalance. A range of losses were tested (BCE variants, Dice/Tversky, focal losses), and it was found that multi-stage training works best in practice: starting with a stable loss (e.g. BCE or BCE+Dice) to establish basic localization, and only later introducing more aggressive imbalance-aware losses (e.g. Focal Tversky) to boost recall on faint trails. This staged approach consistently produced more stable convergence and better object-level performance than single-loss training from scratch.

An important and somewhat unexpected outcome was the discovery of a bug in the LSST stack streak-detection configuration, which caused the classical LSST detection pipeline to perform worse than it should have on injected data. After identifying and fixing this issue, LSST stack performance improved noticeably. This had the side effect of reducing the apparent advantage of the neural-network approach in direct comparisons. This result is scientifically important: it shows that part of the previously observed NN gain was due to a configuration issue rather than a fundamental algorithmic limitation. At the same time, it raises the bar for the ML model and clarifies that future improvements must focus on genuinely hard cases.

Jul 24, 2024 | 📅 LSST Streak Detection Weekly

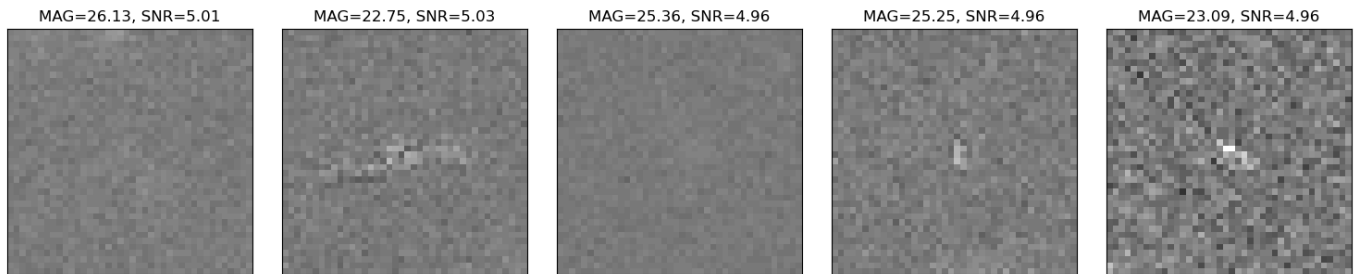
Attendees: Karlo Mrakovcic Chester Li tmc4019@gmail.com

TODO:

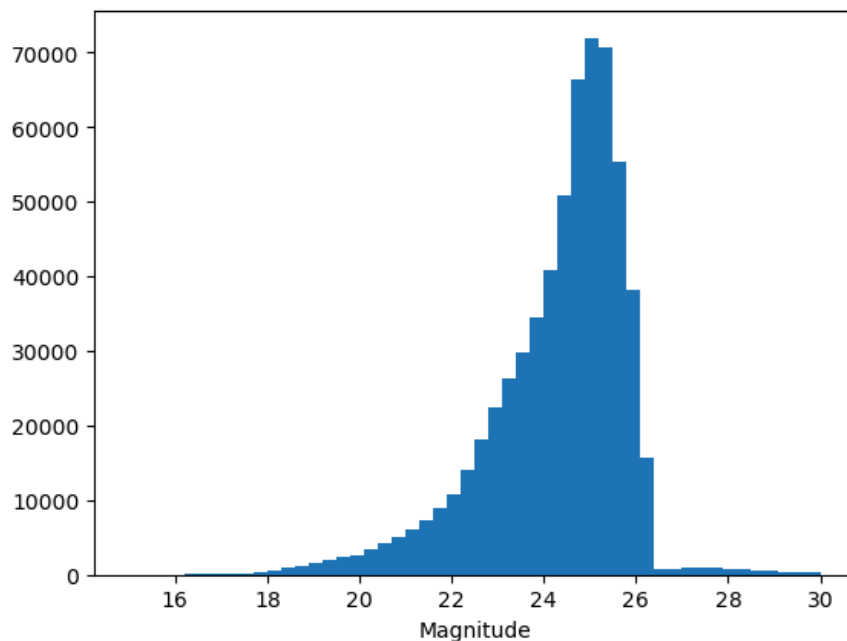
Find all the codes for the attached images in the [notebook](#)

1. Find limiting magnitude

- ☐ Get limiting magnitude information from the header (somewhere in PhotoCalib)
 - **NEED HELP**
- ☒ From detected sources (not asteroids) find a source with $S/N=5$. Calculate its magnitude.

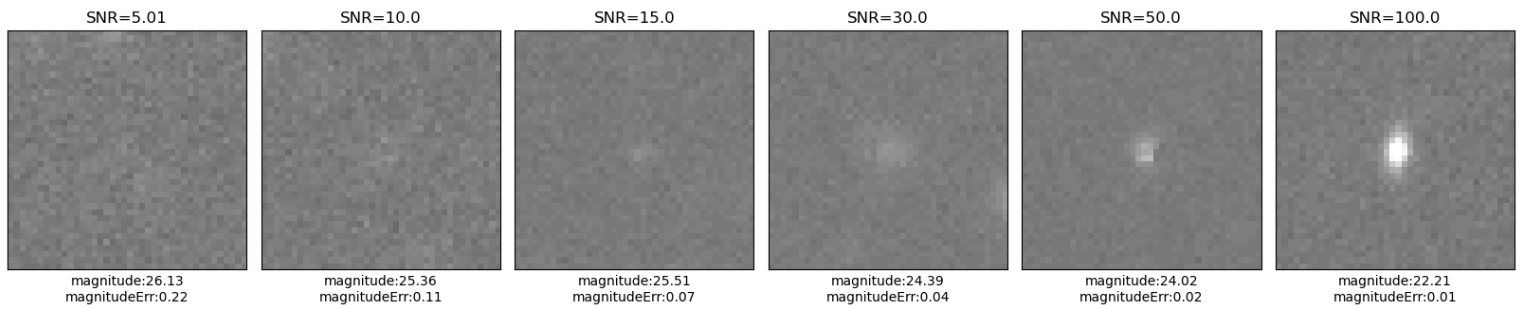


- ☒ Create a histogram of detected sources (not asteroids) and find the cutoff point

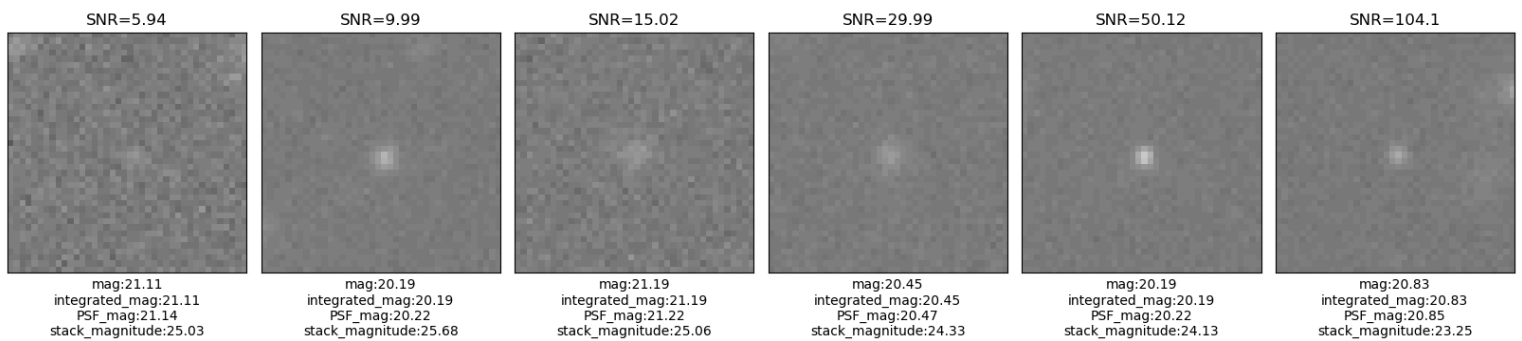


2. Get intuition about plotted asteroids

- ☒ Plot cutouts of detected sources (not asteroids) with different S/N



☒ Plot cutouts of asteroids with only 1 pixel in length



- Keep in mind the limits of [mean-3 sigma, mean+ 3 sigma]

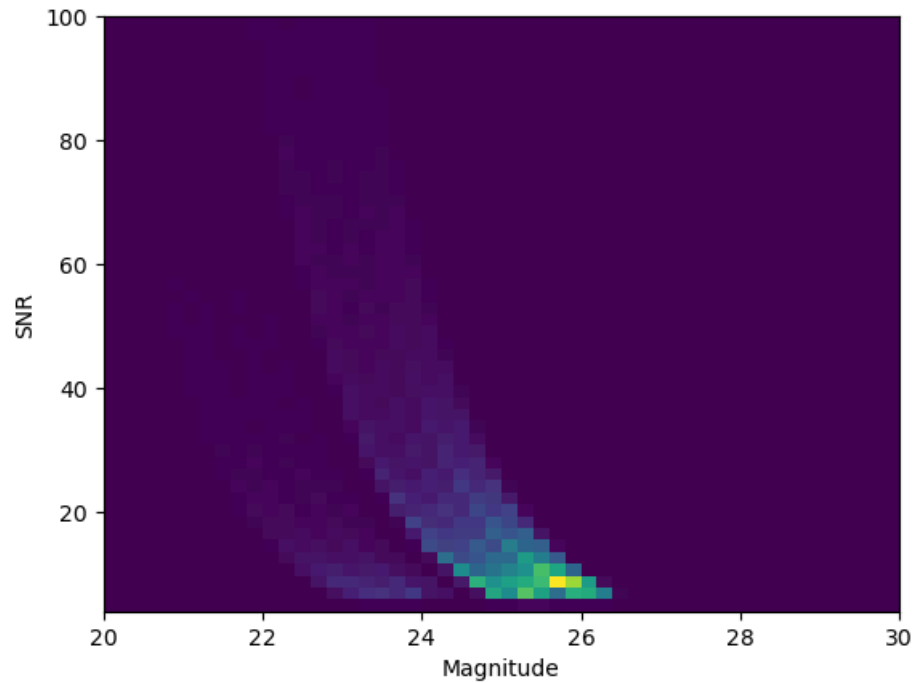
3. Double-check exposure time

☒ Find exposure time info in calexp

```
: calexp.info.getVisitInfo().exposureTime
: 240.0
```

4. Bonus:

- SNR vs Magnitude density plot of sources



- Differences between different magnitudes for asteroids of trail_length=1

	integrated_mag	PSF_mag	surface_brightness	stack_magnitude	stack_snr
10	21.356825	21.381326	21.356825	25.804661	7.252068
12	21.143750	21.168251	21.143750	23.982816	35.367753
16	21.182094	21.206595	21.182094	25.169253	12.718783
23	21.073760	21.095425	21.073760	24.897249	17.383626
24	21.304472	21.326137	21.304472	24.894789	18.019597
...	...	...	...	...	...
4638	20.318846	20.343358	20.318846	24.283679	42.291737
4647	21.414332	21.439305	21.414332	22.692434	77.953141
4660	20.814405	20.839409	20.814405	24.853470	8.125011
4672	20.374736	20.399741	20.374736	24.130807	15.990336
4678	20.196532	20.221536	20.196532	24.147038	15.232050

Catalog creation and model explanation

As I currently understand the matters we are differentiating three types of magnitude:

1. Integrated magnitude: magnitude of the object that appears in the sky. This is what most people think of when saying magnitude
2. Surface brightness: magnitude of the object after its flux was spread out to a streak that appears in the camera. This is because the LSST camera takes 30 seconds of exposure and the asteroids move considerably in that time.
3. PSF magnitude: magnitude after PSF convolution due to atmosphere effects. This dictates the limit of depth to which we can observe.

Another way of understanding this is to imagine you have three telescopes measuring the brightness of the same asteroid. The first one is an ideal telescope in a vacuum with 0 exposure time. The image of this asteroid is a dot and the measured magnitude is the integrated magnitude. Another telescope is also an ideal telescope in a vacuum but has a 30-second exposure time. The image of an asteroid is a perfect line with 0 width and its measured magnitude is a surface brightness. The third one is a pretty good telescope (Rubin), that is in the atmosphere and has a 30-second exposure time. The image is a line that is convolved with PSF and the measured magnitude is a PSF magnitude.

In practice when injecting sources, the input parameter is the surface brightness, as read from the [documentation](#): *"It creates a new object that has the same profile as the original, but with the surface brightness at every location rescaled such that the total flux will be the given value."* That's why we found `object = object.withFlux(inst_flux * source_data["trail_length"])` in [source_inject code](#). On the other hand magnitude limits of LSST described in the [documentation](#) are given in the PSF magnitude terms.

How to properly convert integrated magnitude to PSF magnitude?

The answer lies in [Jones et al.](#) paper:

$$\Delta m = -1.25 \log_{10} \left(1 + \frac{ax^2}{1 + bx} \right)$$
$$x = \frac{vT_{exp}}{24\theta}, \quad a = 0.67, \quad b = 1.16$$

Where v is the velocity (in deg/day), T_{exp} is the exposure time (in seconds), and θ is the FWHM (in arcseconds). Converting vT_{exp} (in deg) to length in pixels l_p , θ from arcsec to θ_p in pixels, and assuming $\Delta m = m_{PSF} - m_I$ (because Δm must be positive and $m_I < m_{PSF}$) where m_I is integrated magnitude and m_{PSF} is PSF magnitude this reduces to:

$$m_{PSF} = m_I + 1.25 \log_{10} \left(1 + \frac{ax^2}{1 + bx} \right)$$
$$x = \frac{l_p}{24\theta_p}, \quad a = 0.67, \quad b = 1.16$$

θ can be taken from this [Table](#) as well as m5 magnitude 9limits.

How does LSST source_inject convert integrated magnitude to surface brightness?

The result of this investigation was described in the following mail:

I am writing as a follow-up on the investigation of how to input correct magnitudes.

After adding `surface_brightness = magnitude + 2.5 * np.log10(inject_length)` in [generate_injection_catalog.py](#) the goal was to verify if we are generating asteroids with expected magnitude behaviour. To do that, I added

```
print ("##### inst_flux * source_data[trail_length]  
= ", inst_flux * source_data["trail_length"])
```

right after line 287 in [inject_engine.py](#) of the source_inject code. This allows me to print the flux in the `pipetask run` command.

Following catalogue for the visit=29336, detector=58 and patch=50 was created:

	ra	dec	source_type	trail_length	mag	beta	visit	integrated_mag
0	149.866489	2.395546	Trail	10.0	23.5	0.0	29336	21.0
1	149.863918	2.393957	Trail	1000.0	28.5	0.0	29336	21.0

Right after that `pipetask run` command was executed where the following lines were found:

```
##### inst_flux * source_data[trail_length] = 81513.7678690941  
##### inst_flux * source_data[trail_length] = 81516.54176068094
```

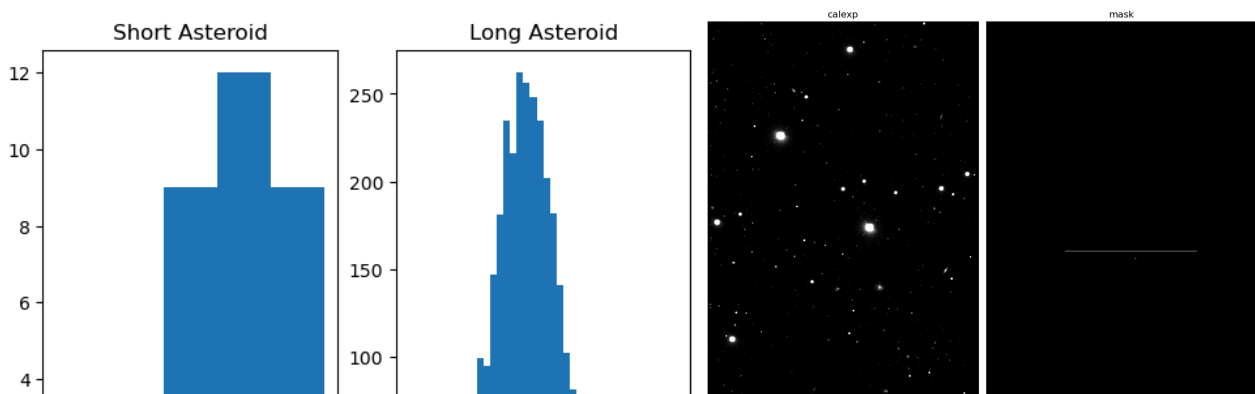
To convert back to magnitude, I traced back the [inject_engine.py method](#) and produced the following line:

```
print (calexp.getPhotoCalib().instFluxToMagnitude(81513.7678690941),  
calexp.getPhotoCalib().instFluxToMagnitude(81516.54176068094))
```

This prints out numbers: 21.005461495423443, 21.00542454874119

Why this is not exactly 21.00 is beyond me, however, it's very very close.

These are the results from this test with median pixel brightness for short asteroid 27.7 and for long asteroid 0.89:



The conclusion is that to transform integrated magnitude to surface brightness we can use:

$$m_{SB} = m_I + 2.5 \log_{10}(l_p)$$

Where m_{SB} is surface brightness, m_I is the integrated magnitude and l_p is asteroid trail length in pixels.

How to produce a source injection catalog for model training?

Code available at [GitHub](#)

1. Roll the dice for asteroid placement in sky coordinates:

```
ra_pos = np.random.uniform(low=min_ra.asDegrees(), high=max_ra.asDegrees())
dec_pos = np.random.uniform(low=min_dec.asDegrees(), high=max_dec.asDegrees())
```

Variables `min_ra`, `max_ra`, `min_dec` and `max_dec` are read from `calexp.wcs` and contain `calexp` borders in sky coordinates.

2. Roll the dice for trail length in pixels:

```
inject_length = np.random.uniform(low=trail_length[0], high=trail_length[1])
```

Variables `trail_length[0]` and `trail_length[1]` are set by the user.

3. Roll the dice for magnitude:

We don't want to create an asteroid so faint that there is no chance any algorithm detects it. The depth limit for PSF magnitude in LSST and PSF FWHM is taken from this [document](#). Therefore when rolling the dice we have to limit the upper bound for the magnitude.

```
a = 0.67
b = 1.16
x = inject_length / (24 * theta_p)
upper_limit_mag = psf_depth - 1.25 * np.log10(1+ (a*x**2)/(1+b*x))
```

After that, we roll the dice for the magnitude and calculate surface brightness from it:

```
magnitude = np.random.uniform(low=mag[0], high=upper_limit_mag)
surface_brightness = magnitude + 2.5 * np.log10(inject_length)
```

4. Roll the dice for angle:

```
angle = np.random.uniform(low=beta[0], high=beta[1])
```

Where `beta[0]` is usually 0 and `beta[1]` is usually 180 degrees.

source_type	trail_length	mag	beta	visit	integrated_mag	PSF_mag
str5	float64	float64	float64	int64	float64	float64
Trail	35.43633134710539	24.247357635368353	23.454247855580753	358	20.373735750622274	21.444775002555843
Trail	11.908943102041881	22.848081697436516	165.38164038331914	358	20.158398646651168	20.72298823023002
Trail	11.825104967538008	24.465542744561454	156.19714366796072	358	21.783530233273574	22.34519161674817
Trail	51.55638690433328	24.829678856897285	167.96787312376264	358	20.548972671428	21.81018797457208
Trail	30.575780693462455	25.117373101436037	89.12806598491979	358	21.403929214635856	22.401720497938953
Trail	38.67495776553578	24.32238860789316	170.53670819699633	358	20.35381398762753	21.46873707129502
Trail	14.679709595996577	23.798902722633976	28.019936229336004	358	20.882109062243742	21.535996418300815
Trail	25.642272915822495	23.678685202675	137.95865797506292	358	20.15629390728098	21.068192081264872
Trail	40.726950794482356	24.972644660395197	25.058385396811897	358	20.947939920639467	22.08895124207195

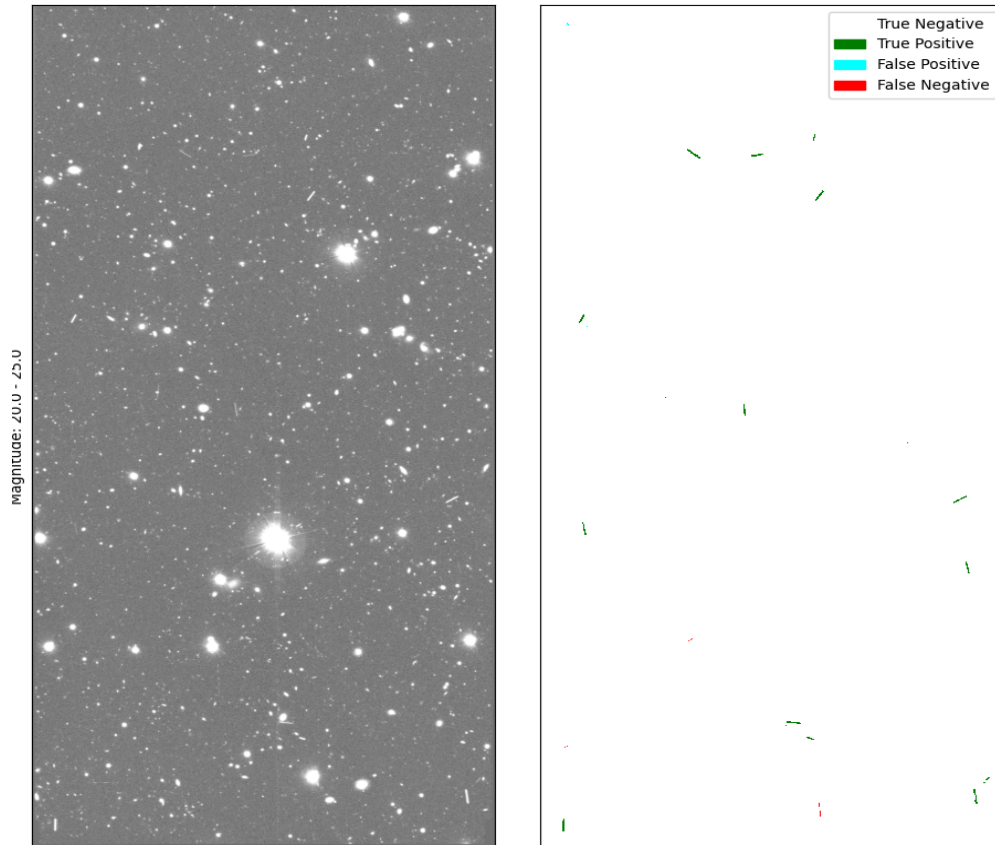
What CNN model are we using?

This is best explained in the following mail:

Starting from the inputs, we are currently using images from the HSC camera, processed by the LSST pipeline with injected asteroids. This is a table (or matrix using linear algebra jargon) of 2048 x 4176 numbers that represent the measured flux of the visible light across the sky. Each of those numbers I call pixels however difference is that pixels are always numbers in the range 0-255, and our flux can be in theory any number. Each image is taken with a different filter in front of the camera so it can be a different colour, however, we aren't concerned with that, we take each image as it was in greyscale. Each image of 2048 x 4176 numbers is cut into 528 smaller 128x128 images and this represents the values of the input layer. This is very similar to the 28x28 input neurons [from the book](#), just the pixels aren't greyscale intensities but the greyscale flux (meaning it's not bound to the 0-255 range). We are not focusing on any particular source, we are using the whole camera panel as it was captured in the telescope.

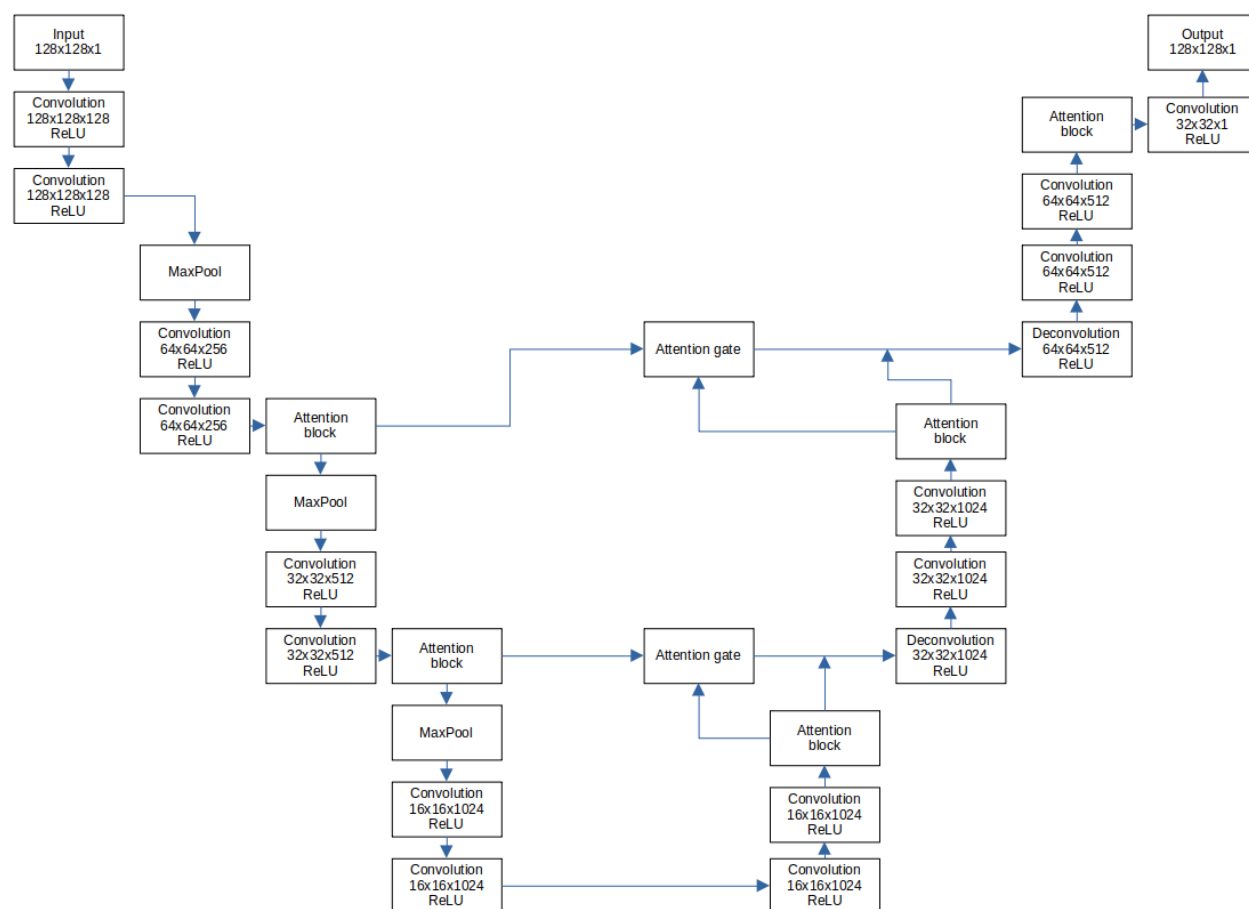
Skipping to the outputs, since we know on which pixels we injected synthetic asteroids, we created a blank 2048x4176 image filled with values 0 at each pixel, except the ones where we added asteroids which have the value 1. This image contains the same amount of pixels as the input image, but each pixel can contain only two values: 1 where the input image has an asteroid and 0 elsewhere. This is what I'm referring to when I say "mask". This mask gets cut in the same way as the input into 128x128 images, and represents the "targets" or sometimes called "labels" of the network. We are training the network to give us the image only containing values 1 or 0; 1 where there is an asteroid on the input image and 0 elsewhere. Note that this is different from the usual approach where the model outputs only one value per image: 1 if the whole image contains an asteroid or 0 otherwise (VGG, ResNet, DenseNet).

In this image you can see the input image (before it was cut into 128x128 smaller images) on the left and the output mask on the right:

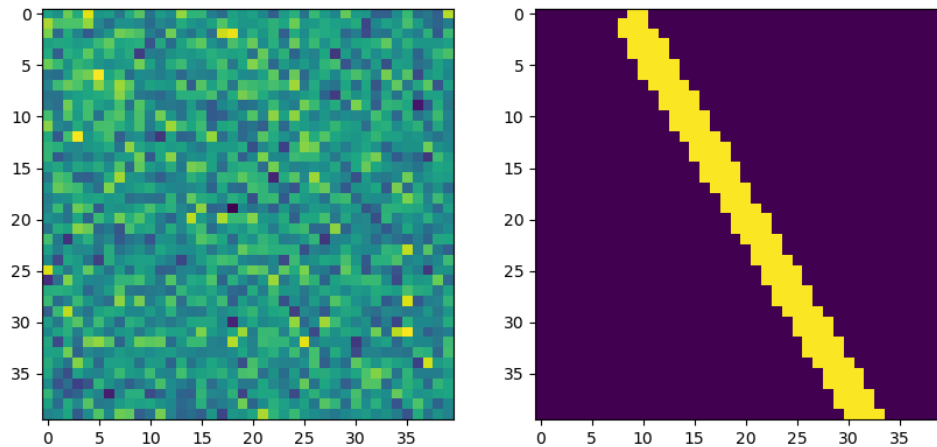


Having these restrictions: input containing grayscale images and output containing images with only 1 and 0, the best approach was to use [U-net](#). The architecture that we are currently using has some additional caveats. From your mail, I understand that you already learned what convolution is. This is a basic building block of the network. However additional types of building blocks are used as well. One of them is the Attention block described in the paper "[CBAM: Convolutional Block Attention Module](#)". The best analogue for this block is human attention, meaning it controls which part of the image network is focused more on. This block contains weights and biases that are learned, meaning we are not giving preference to any region of the images by hand. Another specific part is called the Attention gate described in the paper "[Attention U-Net: Learning Where to Look for the Pancreas](#)". This block is added to the skip connections, and its goal is to allow the network to learn which neurons of the skip connections are more important than the others. This way we are trying to propagate only relevant information through skip connections (skip connections are the ones going through the middle). This current architecture is the product of experimentation and there is no convincing explanation for why this amount of hidden layers works currently the best.

Here is a simple sketch of the current model we are using:



Another part of the model training is using the loss function. Usually, simple models use ["Mean square error"](#) as a loss function which is an analytic maximum likelihood solution for normal Gaussian distribution assuming constant variance. This means that this loss function works best when there is a similar amount of "positive" and "negative" classes in the output. In our example, amount of positive classes is the number of pixels in the output mask with the value 1 (asteroids) and negative classes are pixels with the value 0 (everything else). It's easy to see that the number of pixels containing asteroids is far less than the others. Ratio is about 99.99% non-asteroid pixels and 0.01% asteroids. This means we have a highly imbalanced dataset, and we need to use special loss functions to account for that. Currently, we are using the Focal Twersky loss function described in the paper: ["A Novel Focal Tversky loss function with improved Attention U-Net for lesion segmentation"](#). The parameter alpha is set to 0.9 and the parameter gamma is set to 3. Currently, we are starting with the learning rate of 0.001, but reducing it by a factor of 0.75 once the loss function reaches a plateau. This loss function was chosen for its robustness over the class imbalance as well as the ability to tune parameters alpha and gamma to maximize completeness.



Jun 26, 2024 | 📅 LSST Streak Detection Weekly

Attendees: Karlo Mrakovcic Chester Li tmc4019@gmail.com

Notes

- We have a problem regarding the surface brightness of the injected sources. I have attached an image to this email. The injected streaks are visible on the right panels, but they are nowhere to be found on the images on the left. The CNN is not able to detect the streaks. The reason might be that the streaks' surface brightness (about 28, 29 mag) is too low for the CNN to detect.
- On the other hand, a streak with a surface brightness of 23.5 mag is barely visible in another image. Should we only generate streaks with surface brightness brighter than 23.5 mag so that they can be recognized by the CNN? Or do you have more suggestions?

May 22, 2024 | 📅 LSST Streak Detection Weekly

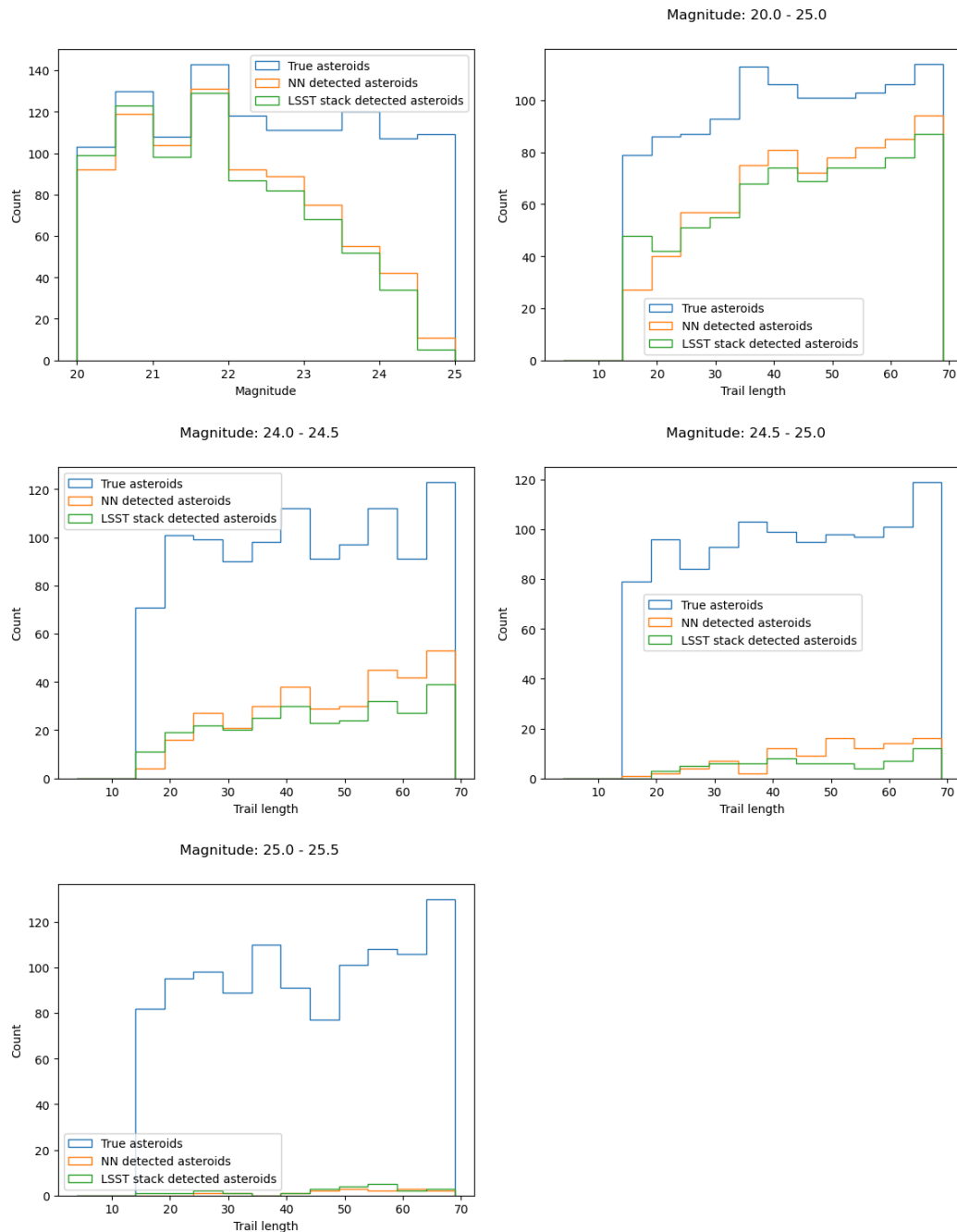
Attendees: Karlo Mrakovcic Chester Li Guinevere Berg

Notes

- Chester got access to data on SLAC & confirmation that we are allowed to use it now
 - Chester to install LSST pipeline & tensorflow, clone the [github repo](#)
 - Karlo needs info on where the Butler repository is on SLAC
 - Karlo will adjust the injection script to run on SLAC
- Next week: figure out how to transfer data to HYAK

May 8, 2024 || 📅 LSST Streak Detection Weekly

Notes:



- Hyperparameter tune finished, basic model performs acceptably

- Experimentation with loss ([focal twersky](#))
- Next step: attention block + downscaling of the output + deeper network
- Write a new metric that tells us what amount of objects did we miss (F1 score but objectwise)
- Mario:
 - Some auxtel data with asteroids available at <https://roundtable.lsst.codes/rubintv-dev/summit/auxtel/historical/2024-02-15>

Apr 10, 2024 || 📅 LSST Streak Detection Weekly

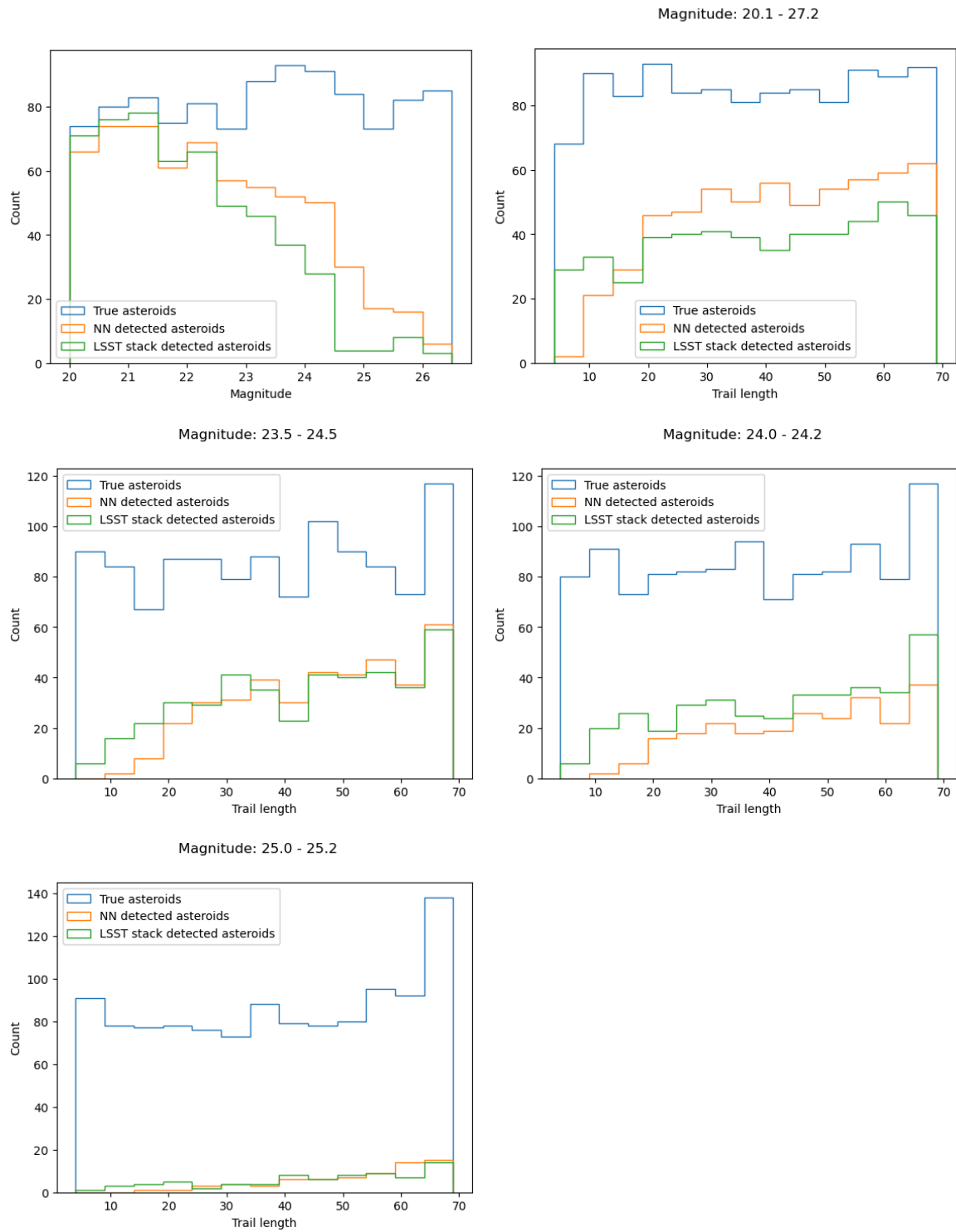
Notes:

- Chester
 - Waiting for SLAC approval
 - Found processed HSC data on the Rubin Science Platform
 - Will copy the data to Hyak
 - Will meet with Guinevere this week to talk about Hyak
-

Apr 10, 2024 || 📅 LSST Streak Detection Weekly

Notes:

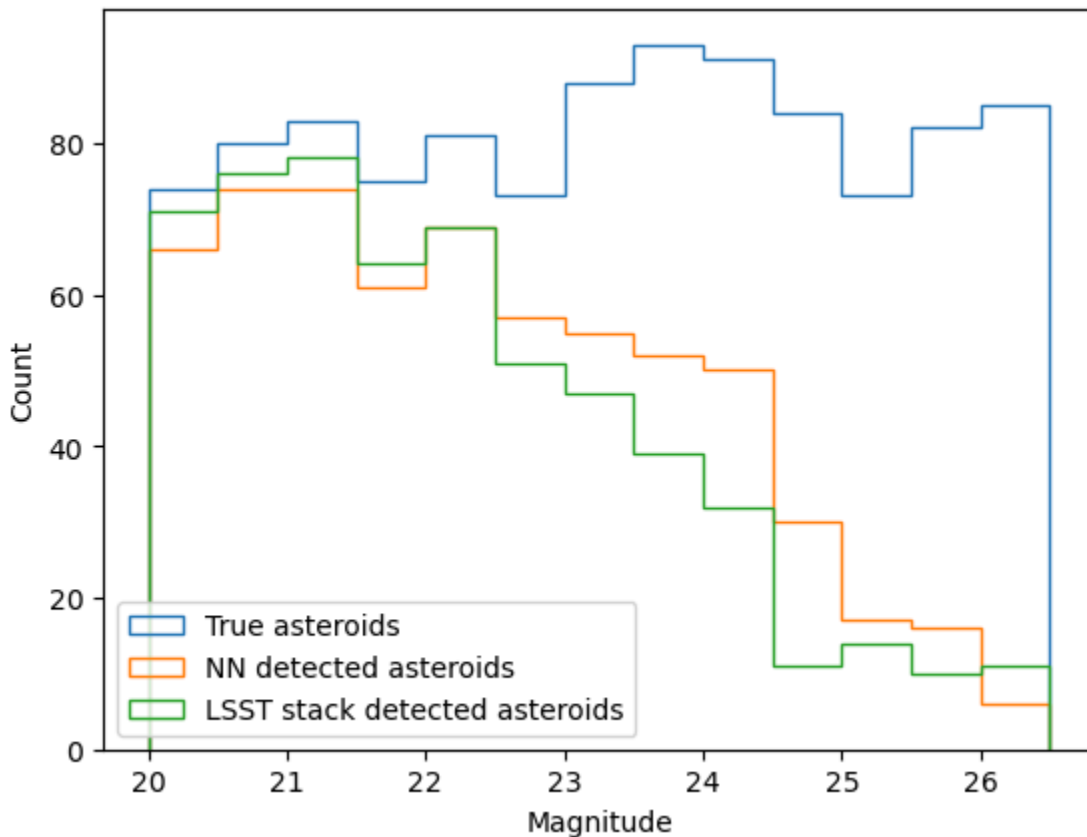
- Hyak
 - LSST stack installed
 - Source injection package installed
- To do
 - Mario: Set up a meeting with Dima Duev to show what we're doing and ask for comments
 - How much HSC data do we need specifically?
 - 3 times of 9 GB
 - Different star fields
 - As much of the sky coverage as possible
 - Same patch of the sky with different injections x 2
 - Chester's Rubin commissioning team access
 - Guinevere's Hyak access



Apr 3, 2024 || 📅 LSST Streak Detection Weekly

Notes:

- TransiNet paper: <https://arxiv.org/abs/1710.01422>
- Karlo needs GPU hardware that can run Tensorflow for hyperparameter search
 - Chester's to-do list on Hyak (<https://hyak.uw.edu/>)
 - Install Rubin pipeline
 - Get more HSC data + simulated streaks
- Detection efficiency as a function of magnitude:



Mar 27, 2024 ||  LSST Streak Detection Weekly

Notes:

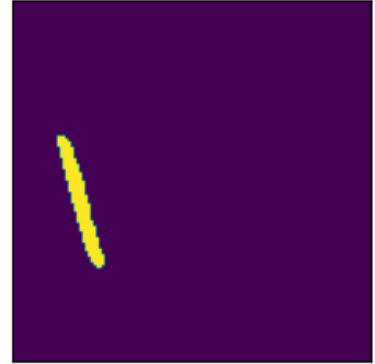
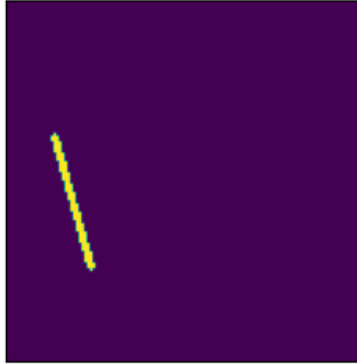
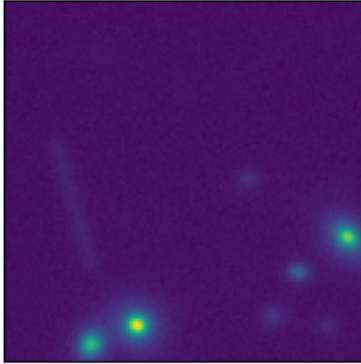
- GitHub repo: https://github.com/dirac-institute/Asteroid_detection_CNN.git
- Papers:
 - Detecting solar system objects with convolutional neural networks: <https://arxiv.org/pdf/1807.10912.pdf>
 - <https://arxiv.org/abs/1904.05920>
 - <https://ui.adsabs.harvard.edu/abs/2022MNRAS.516.5785W/abstract>
- Trying to figure out asteroid-wise metrics instead of pixel-wise
- Create a histogram of predicted asteroids dependence of magnitude
- Generate up to 25.5 mag

Mar 6, 2024 || 📅 LSST Streak Detection Weekly

Attendees: Mario Juric mrakovcickarlo@gmail.com zhuofu@uw.edu
tmc4019@gmail.com Guienevere

Notes:

- Chester: need to get into the eScience group on Klone (to get access to GPUs)
- Mario: 100TB of space on hyak



Feb 21, 2024 || 📅 LSST Streak Detection Weekly

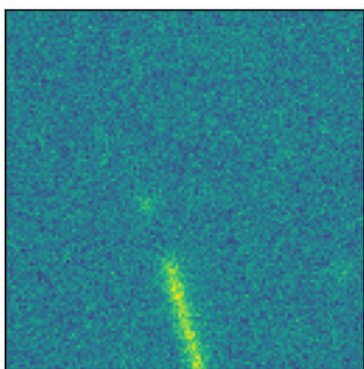
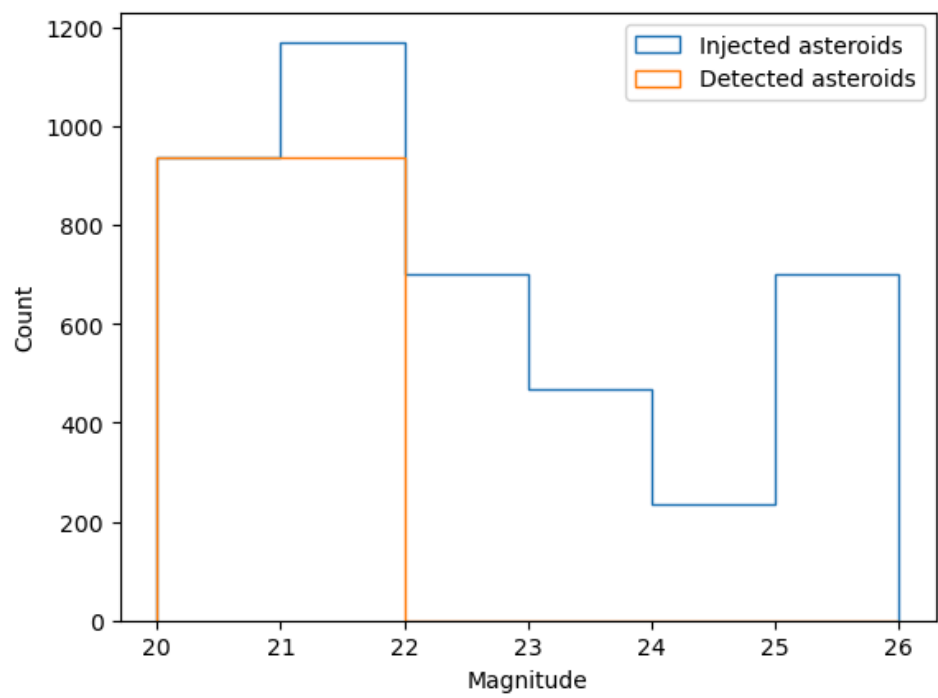
Attendees: Mario Juric mrakovcickarlo@gmail.com zhuofu@uw.edu

Notes:

-

Action items:

- Karlo
 - Start to train
 - Make coadd
- Chester
 - Figure out how to run injection using Karlo's code
 - Run on Hyak

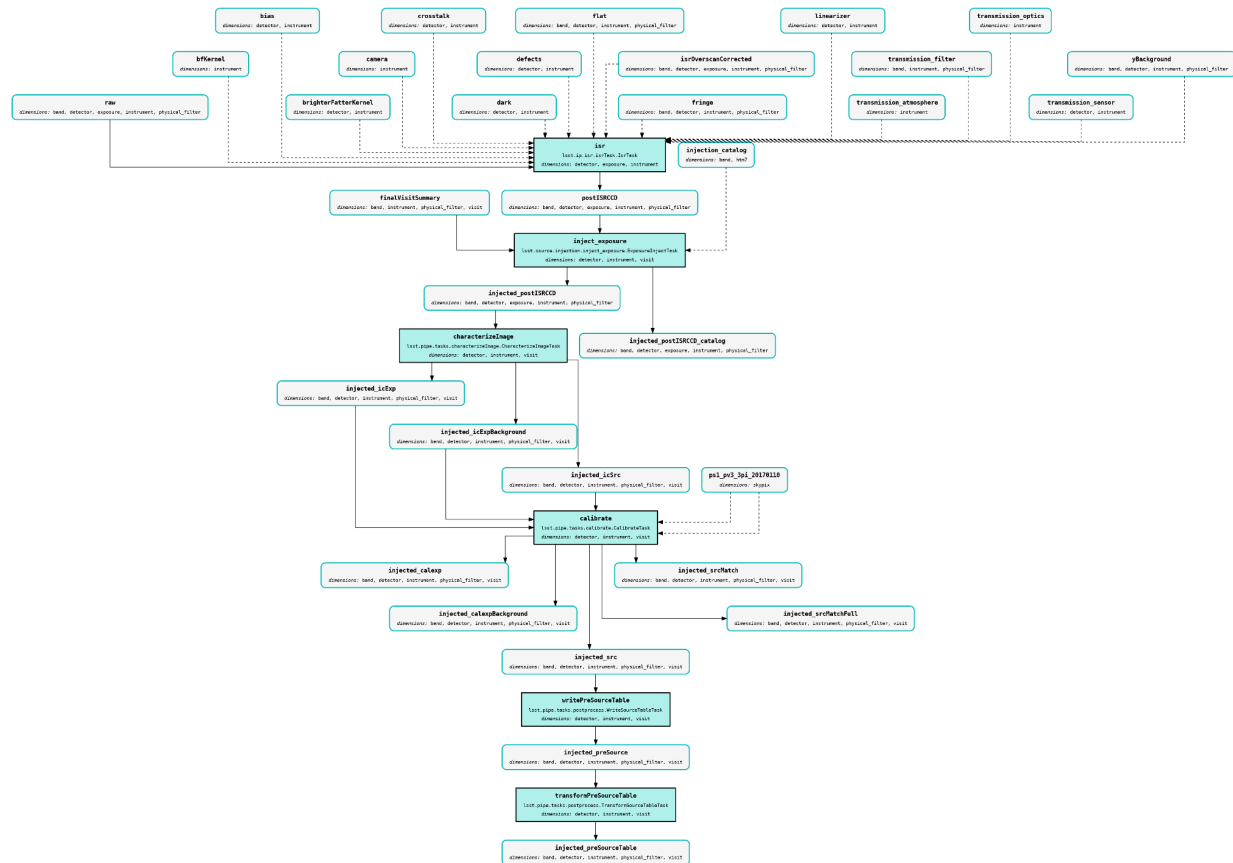


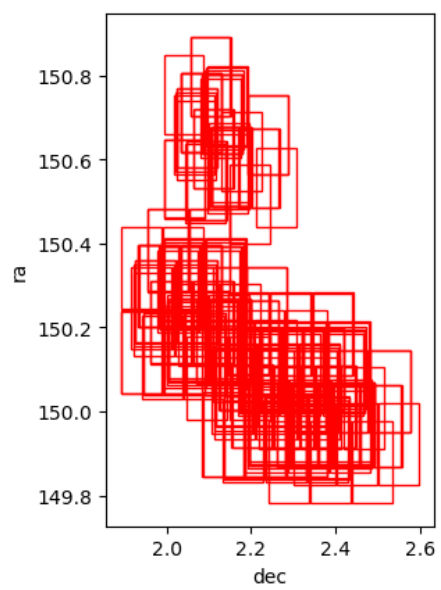
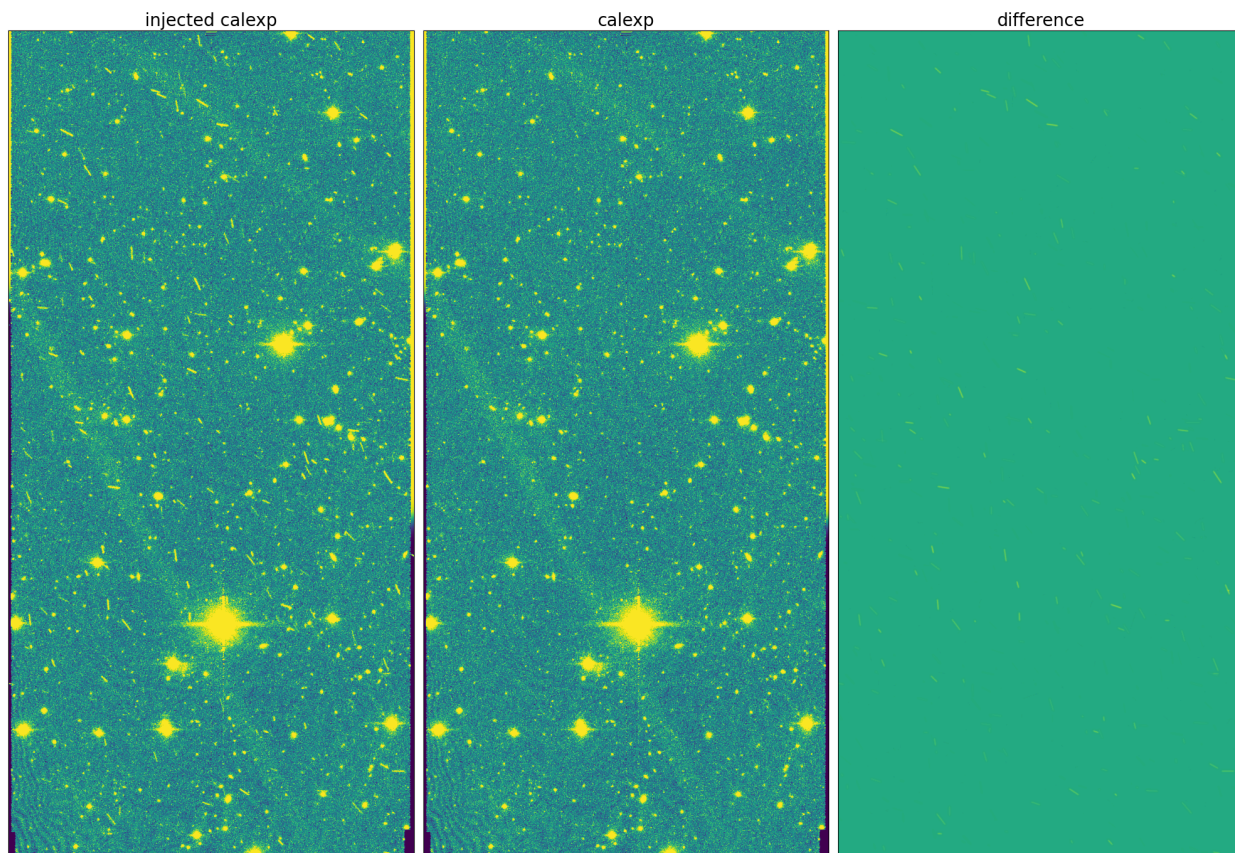
Feb 14, 2024 || LSST Streak Detection Weekly

Attendees: Mario Juric mrakovcickarlo@gmail.com zhuofu@uw.edu
tmc4019@gmail.com

Notes:

- Chester: fixed the installation problem. Have all now packages running successfully and have obtained the data. Playing with the injections now
 - Get ready to run at large scale
 - Try to produce a row of trails with different magnitudes
- We'll need a larger chunk of HSC data, imported into the Butler.
- Problem: sources don't have time information in the injection catalog – they're always there! We'll need to add it somehow.
 - https://github.com/lst/source_injection/blob/6ab0d2389a386bb72cd42e8d0e76c1e91a89e3ad/python/lst/source/injection/inject_engine.py#L170
 - https://github.com/lst/source_injection/blob/6ab0d2389a386bb72cd42e8d0e76c1e91a89e3ad/python/lst/source/injection/inject_base.py#L166
 - Idea:
 - Add a visitId field to
 - eups setup -r





Feb 7, 2024 | 📅 LSST Streak Detection Weekly

Attendees: Mario Juric mrakovcickarlo@gmail.com zhuofu@uw.edu

Notes:

- Inject asteroids with uniform distributions: 74 - 4 px in length, 20 - 27 magnitude, $N = 20$ per detector, all angles.
- Chester: Problems with the simulation package generating errors on UW's cluster. We don't understand what the errors are telling us. Scheduling another meeting with Lee (it's taking time).
- Karlo: Injection works (on epyc), in a notebook! Showed a nice demonstration.

Action items:

- ☐ Inject various lengths from 0.5 - 10 degrees
- ☐ Run source detection
 - ☐ Confirm the plot from Lynne's paper

Jan 24, 2024 | 📅 LSST Streak Detection Weekly

Attendees: Mario Juric mrakovcickarlo@gmail.com tmc4019@gmail.com zhuofu@uw.edu

Notes:

- Karlo and Chester met with Lee Kelvin
- Karlo created the injection pipeline on Epyc
- Chester got instructions from Steven Stelzer on creating Kernel that runs the injection package

Action items:

- ☒ ~~Use the injection package to make a trail of width 0 and compare it to a star~~
It is a 1e-6 pixel-wide line by default (this is OK):
https://github.com/lsst/source_injection/blob/b5039a9374afbe5673ed5db4a0aecf2f9be9ad77/python/lsst/source/injection/inject_engine.py#L248
- ☐ Meet with Lee

Jan 17, 2024 | 📅 LSST Streak Detection Weekly

Attendees: Mario Juric mrakovcickarlo@gmail.com tmc4019@gmail.com zhuofu@uw.edu

Notes:

- Stack installed on bura and epyc, and datasets located (see the notes from previous meeting)
- Decision: we'll focus on the HSC dataset initially for development (use the https://github.com/lsst/testdata-ci-hsc_subset)
- **KM update:** we will focus on https://github.com/lsst/rc2_subset
- Next: figure out how to do source injection, connect with Lee Kelvin to figure out the details
- Chester & Karlo will read the docs for source injection in preparation for talking to Lee

Action items:

- ☐ Mario
 - ☐ Connect the team with Lee Kelvin
- ☐ Chester, Karlo
 - ☐ Karlo:
 - ☒ ~~Try running HSC demo on epyc (ensure it runs)~~
 - ☐ Read up on running the simulation code
 - ☐ Run the demo from the documentation
 - ☐ Inject stars
 - ☐ Inject boxes
 - ☐ Try to set the width of the box to 0
 - ☐ Set the orientation and length of the boxes

Jan 10, 2024 | 📅 LSST Streak Detection Weekly

Attendees: Mario Juric mrakovcickarlo@gmail.com tmc4019@gmail.com zhuofu@uw.edu

Notes:

- We have the object injection code from the LSST stack:
- TODO: Where is the HSC data?
- Some helpful computing (and other) documentation: <https://dirac.us/intro>
- `ssh epyc.astro.washington.edu`

KM update (13 Jan 2024):

- To install the LSST stack I found this: <https://pipelines.lsst.io/v/daily/install/lsstinstall.html>
- We got some written instructions from Steven on how to install the LSST stack
- This is what I did to install the LSST pipeline on Bura:

```
mkdir -p lsst_stack
cd lsst_stack
# get script to install LSST Science Pipelines
curl -OL https://ls.st/lsstinstall
```

```

chmod u+x lsstinstall
# install LSST Science Pipelines into an existing conda environment
./lsstinstall -p ../anaconda3 # if there is no anaconda installed, omit "-p
../anaconda3"
# Load LSST software environment
source loadLSST.sh
# install science pipelines
eups distrib install -t w_latest lsst_distrib
curl -sSL
https://raw.githubusercontent.com/lsst/shebangtron/main/shebangtron |
python
# Load science pipelines
setup lsst_distrib

```

- This is what I did to install source_injection

```

# Load science pipelines
setup lsst_distrib
git clone https://github.com/lsst/source_injection.git
cd source_injection
setup -k -r . -t $USER
scons -Q -j 8

```

- To activate it

```

setup lsst_distrib
cd /some/path/to/source_injection
setup -k -r .

```

- HSC RC2 dataset: [Getting started tutorial part 1: setting up the Butler data repository — LSST Science Pipelines](#)
- HSC camera basic data & layout visualization: [Basic Information for Hyper Suprime-Cam — hscpipe e 0.0 documentation](#)

Chester's Updates (01/17/24):

- Installed the synthetic source injection package on Epyc
- Found data
 - DECam Data: raw data on epyc are at /epyc/data3/DEEP/raw
 - HSC Data: https://github.com/lsst/testdata_ci_hsc/

```
[zhuofu@epyc ~]$ bash
bash-4.2$ source /epyc/ssd/users/stevengs/opt_lsst/bin/opt_lsst.sh
bash-4.2$ $ opt_lsst list # list available versions
bash: $: command not found
bash-4.2$ opt_lsst list # list available versions
d_2024_01_15
d_2024_01_16
d_latest
v26_0_0_rc1
w_2023_38
w_2024_01
w_2024_02
w_latest
bash-4.2$ opt_lsst setup w_latest
w_latest -> w_2024_02
loading version w_2024_02 of the LSST Pipelines
(lsst-scipipe-8.0.0) bash-4.2$ ls
Desktop Downloads NEO_FollowUp Public Trojan Asteroids pipelines
Documents Music Pictures Templates Videos source_injection
(lsst-scipipe-8.0.0) bash-4.2$ cd source_injection/
(lsst-scipipe-8.0.0) bash-4.2$ cd source_injection/
(lsst-scipipe-8.0.0) bash-4.2$ setup -k -r .
(lsst-scipipe-8.0.0) bash-4.2$ python
Python 3.11.7 | packaged by conda-forge | (main, Dec 15 2023, 08:38:37) [GCC 12.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import lsst.source.injection
>>>
```

Action items:

- ☐ Chester, Karlo
 - ☒ Meet with Steven/Dino to install the LSST stack
 - ☒ Find the DECAM data
 - ☐ Then run the simulation code
 - ☐ Inject stars
 - ☐ Inject boxes
 - ☐ Try to set the width of the box to 0
 - ☐ Set the orientation and length of the boxes
- ☒ Mario
 - ☒ Find HSC data

Jan 3, 2024 | 📅 LSST Streak Detection Weekly

Attendees: Mario Juric mrakovcickarlo@gmail.com tmc4019@gmail.com zhuofu@uw.edu

Notes

- Welcome everyone!
- The mission: prototype an ML-based code for near-optimal streak detection in Rubin images.
- Details: Rubin's source detection algorithm is currently optimized for searching for point sources. Faint objects that move (and therefore "smear" their available flux along a streak) will therefore be harder to detect. This is quantified nicely in Figure XX of [Jones et al.](#):

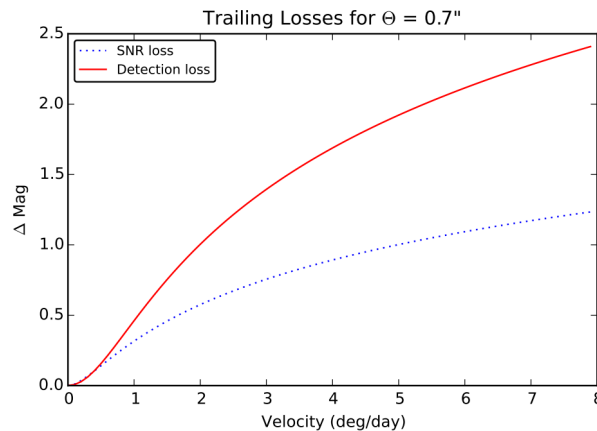


Fig. 6.— Trailing losses for 30 second LSST visits, assuming seeing of $0.7''$. The dotted line shows SNR trailing losses, the solid line indicates cumulative losses that also account for non-optimal detection algorithm. With software improvements the latter detection losses can be mitigated. At the fiducial $v = 1$ deg/day, the SNR loss is ~ 0.3 mag, and non-optimal detection algorithm contributes an additional loss of ~ 0.16 mag.

An optimal detection algorithm is represented by the dotted line; Rubin's current detection algorithm is the red line. At large velocities (say, 8 deg/day), we lose nearly 1.75 magnitudes of sensitivity because we don't have an optimal algorithm. So another restatement of our mission is to try to develop something that gets us as close to the blue line as possible.

- First step: we need a good training/test dataset, with a known ground truth. We'll generate it ourselves, by injecting streaks into real images from HSC or DECam. This way we can generate as much training data as we need.
- **MJ update (Jan 5, 2024):** the software to use is https://github.com/lst/source_injection. The documentation appears relatively good. The main author is [Lee Kelvin](#) (I've interacted with Lee a few times, he's really nice – feel free to reach out to him for help w. setting the code up or how to run it).

Action items

- ☒ ~~Mario Juric~~ : ~~Locate the software to inject fake sources into images.~~
- ☐ zhuofu@uw.edu mrakovcickarlo@gmail.com : Once Mario provides the lead, Chester should try installing it (ideally on epyc) and figure out how to run it.